

Your Vision-Language-Action Model Already Has Attention Heads For Path Deviation Detection

Jaehwan Jeong^{1,2}, Evelyn Zhu², Jinying Lin², Emmanuel Jaimes², Tuan-Anh Vu², Jungseock Joo^{2,3}, Sangpil Kim^{1,†}, and M. Khalid Jawed^{2,†}

¹Korea University ²University of California, Los Angeles ³NVIDIA

Abstract. Vision-Language-Action (VLA) models have demonstrated strong potential in navigation tasks by predicting semantic actions and reasoning over complex linguistic and visual contexts. However, they are fundamentally hindered by visual-reasoning hallucinations causing trajectory deviations. Addressing this issue has conventionally required training external critic modules or relying on complex uncertainty heuristics. In this work, we discover that monitoring specific attention heads within a frozen VLA model detects path deviations without incurring additional computational overhead. We term these Navigation Heads, as they inherently capture the spatiotemporal correlations between historical visual sequences and linguistic instructions. Using these heads, we propose an intuitive, training-free anomaly-detection framework that monitors their signals to detect hallucinations in real time. Surprisingly, among over a thousand attention heads, a combination of just three is sufficient to achieve a 65.1 % anomaly detection rate (recall) and a 76.4 % F1 score in unseen environments. Furthermore, we couple this high-level detection logic with a low-level Reinforcement Learning (RL) policy for local obstacle avoidance, ensuring stable navigation and executing a direct safe recovery upon anomaly detection. Ultimately, deploying this entire framework zero-shot onto our customized physical robot demonstrates its practical robustness. All source code will be publicly available.

Keywords: Vision-Language Action (VLA) · Reinforcement Learning (RL) · Navigation Path Recovery · Robot Operating System (ROS)

1 Introduction

Driven by recent advances in Large Vision-Language Models (LVLMs), vision-language navigation has progressed from discrete graph-based agents to Transformer based policies that can reason over long instructions and rich visual observations [3, 7, 8, 15, 17, 51]. More recently, Vision-Language-Action (VLA) models have extended this paradigm to continuous environments by directly mapping sequential visual observations and language instructions to executable navigation actions [28]. This formulation is attractive because it removes the need for predefined topological constraints and enables agents to operate in open-world settings using unified multimodal reasoning [10, 38, 55, 58].

[†] Co-corresponding authors.

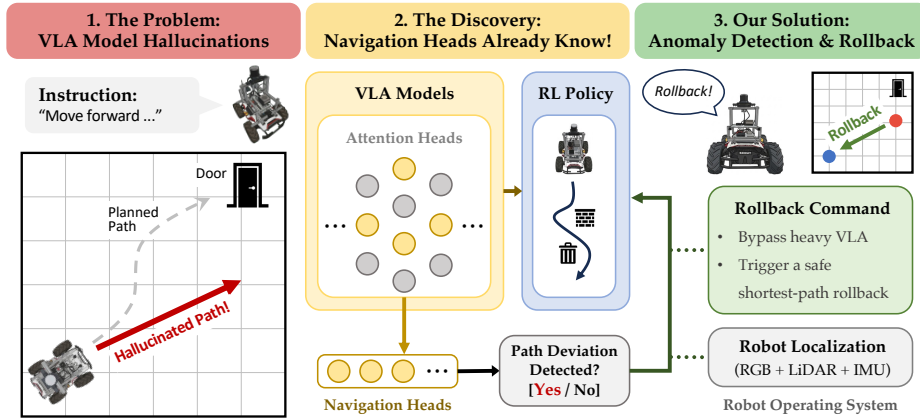


Fig. 1: Conceptual teaser of our approach to mitigating hallucinations in VLA-based navigation. **1. The Problem:** VLA models are highly vulnerable to vision-language reasoning hallucinations, which cause severe deviations from planned trajectories. **2. The Discovery:** We reveal that a specific subset of internal attention heads, termed Navigation Heads (H_{nav}), is highly sensitive to navigation states. This allows them to serve as a built-in anomaly detector with near-zero computational overhead. **3. Our Solution:** We enable real-time, training-free path deviation detection. Upon detecting a failure, the system immediately bypasses the VLA model and triggers a low-level Reinforcement Learning (RL) policy to execute a safe, collision-free rollback.

Although VLA models have demonstrated significant potential for predicting semantic actions, they are fundamentally limited by inheriting the chronic hallucination issues of their Large Language Model (LLM) backbones. As the agent accumulates longer visual histories while following multi-step natural-language instructions, failures in visual reasoning and language grounding can lead to trajectory drift, incorrect sub-goal selection, and ultimately catastrophic path deviations [9, 52]. Prior work has highlighted the importance of detecting uncertainty and initiating backtracking when an agent loses its path [1, 5, 35, 54].

However, current path deviation detection logic primarily relies on simple heuristics like post-facto deadlocks (e.g., physical collisions or position stagnation) [1, 18, 56], requires expensive dedicated fine-tuning of auxiliary networks [22, 34, 50], or invokes computationally heavy external LLMs via APIs [19, 45]. Consequently, these methods fail to provide efficient, high-frequency monitoring and are inherently prohibitive for real-time deployment.

In this work, we ask a different question: Does a frozen VLA model already contain internal mechanisms that reveal whether navigation remains properly grounded? We show that the answer is yes. Specifically, we find that a small subset of attention heads consistently aligns spatiotemporal visual observations with the relevant segments of the navigation instruction. We refer to these heads as Navigation Heads (H_{nav}). During successful navigation, these heads exhibit a structured attention progression that tracks the agent’s movement through the instruction over time. As the agent deviates, this structure degrades in characteristic ways. This observation provides a reliable anomaly indicator for the model’s

internal navigation process and suggests that grounding failures can be detected directly from the VLA model itself, without training an auxiliary detector.

Building on this insight, we propose a training-free spatiotemporal grounding diagnostic framework that formalizes H_{nav} dynamics, ensuring model-agnostic [10, 58] and data-driven robustness by evaluating parameters optimized solely on each model’s training split zero-shot across validation splits. As illustrated in Fig. 1, by monitoring the attention dynamics of these heads in real time, our method produces a continuous signal of the agent’s perceptual grounding state without additional trainable parameters. This enables the system to identify anomalous trajectories as they emerge and seamlessly roll back to the last verified safe state. Notably, our approach is directly linked to a robust anomaly indicator derived from the model’s internal computations, enabling effective real-time anomaly detection without incurring additional computational overhead.

Furthermore, to overcome the slow inference speeds inherent to massive VLAs [6, 12, 13, 27, 33, 49], we develop a custom low-level controller based on Reinforcement Learning (RL) trained specifically for obstacle avoidance. By integrating this lightweight RL controller with the high-level VLA model, our framework ensures stable navigation under normal conditions and guarantees direct safe recovery maneuvers with local obstacle avoidance upon anomalous behavior detection. This hierarchical integration effectively bridges the gap between computationally heavy visual reasoning and reactive physical execution, thereby securing real-world applicability in dynamic environments. Finally, by implementing this integrated system directly on the device, we validate the practicality and robustness of our proposed architecture through extensive evaluations in both the VLN-CE virtual environment and real physical robot deployments. To the best of our knowledge, we are the first to identify navigation-specific attention heads and successfully leverage them for real-world robotic applications.

To summarize, our main contributions are as follows:

- We identify a small subset of attention heads in a frozen LVLM that consistently aligns spatiotemporal visual observations with linguistic navigation intent. We term these heads Navigation Heads (H_{nav}) and show that they serve as a core anomaly indicator underlying language-guided navigation.
- We propose a training-free anomaly detection framework that formalizes the structural evolution and breakdown of H_{nav} during navigation. By monitoring attention dynamics online using only three heads, our method estimates the agent’s navigation state in real time without introducing any additional trainable parameters, achieving a 65.1 % anomaly detection rate (recall) and a 76.4 % F1 score in R2R unseen environments.
- We integrate the anomaly-detection signal into a lightweight RL controller for reactive obstacle avoidance, thereby establishing a robust navigation-and-recovery pipeline. This integration ensures stable navigation and supports a direct safe recovery behavior when an anomalous trajectory is detected.
- We validate the proposed system in real-world robotic environments, demonstrating that our framework is highly practical beyond simulation and performs reliable embodied navigation under real deployment conditions.

2 Related Work

Motion-based heuristics detection. As a fundamental approach, this method relies on rule-based mechanisms to detect a post-facto deadlock, such as physical collisions or position stagnation. Beginning with the DWA [14], which detects entrapment in local minima, subsequent robotic systems have utilized various hardware inputs, including wheel odometry [39], simple sensor thresholds [18], and multi-modal sensor fusion [56], to reactively identify physical entrapments and execute recovery behaviors like map initialization or in-place rotations. This same logic has been recently applied to VLN. For example, ETPNav [1] registers a deadlock if the agent’s coordinates remain unchanged after a forward action, subsequently proposing an in-place rotation heuristic.

Learning-based auxiliary adapters. To proactively detect trajectory deviations, recent paradigms rely on learning-based auxiliary adapters that necessitate training additional detection modules alongside the main navigation backbone. For instance, the Self-Monitoring Navigation Agent (SMNA) [34] introduces a progress monitor module trained with an explicit loss function to estimate goal proximity. Similarly, Speaker-Follower models [15] train a separate speaker network to score the alignment between candidate trajectories and instructions, while IEDL [50] trains a cross-modal transformer specifically to detect and localize instruction-execution errors. In physical robotics, models like PAAD [22] learn deep neural networks to predict future failure probabilities by combining planned motions and perception data. Other approaches, such as BADGR [23] and LaND [24], leverage self-supervised learning or human disengagement data to train predictive models for upcoming collisions. Furthermore, methods like SCoA [61] train a separate reinforcement learning policy to request external assistance when path uncertainty increases. Additionally, architectures such as LSTM-VAE [44] and LSTM encoder-decoders [37] are trained to detect anomalies by measuring reconstruction errors on multi-modal sensor streams. Consequently, these methods fundamentally require not only expensive fine-tuning of auxiliary networks, but also additional computational overhead during inference.

Training-free detection via external models. While avoiding additional fine-tuning, another line of research relies on invoking computationally heavy external Large Language Models (LLMs) or Vision-Language Models (VLMs) to reason about current states and handle path deviations in a zero-shot manner. For instance, hierarchical dual-system frameworks [19] leverage a massive VLM (System 2) to decompose instructions and guide a low-level control policy (System 1), but often suffer from latency and a lack of mutual awareness during execution errors [31]. More explicitly targeting navigation failures, frameworks such as SayNav [45] dynamically convert the agent’s accumulated scene graphs into textual prompts, querying an external LLM to determine whether the current trajectory has deviated and to refine the plan in case of execution failures. However, despite being training-free, utilizing these massive external models as high-level error detectors necessitates auto-regressive text generation via APIs, which inherently introduces severe inference latency.

3 Method

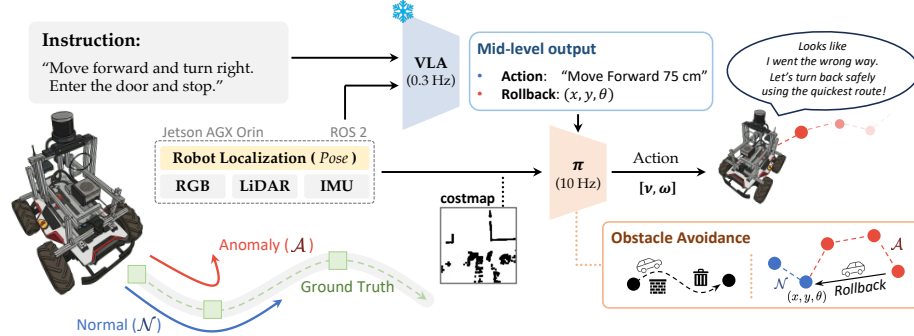


Fig. 2: Overview of the path recovery navigation framework. Deployed via ROS 2, our hierarchical system integrates a high-level VLA model (0.3 Hz) and a low-level RL policy (10 Hz). Using RGB, instructions, and pose data, the VLA detects real-time path deviations ($\mathcal{N} \rightarrow \mathcal{A}$) and triggers a rollback to the last verified checkpoint (x, y, θ) . Concurrently, the RL policy (π) utilizes LiDAR costmaps to output collision-free velocity commands $[v, \omega]$. This design enables the robot to dynamically avoid obstacles and safely return to the normal path via the quickest route during recovery.

3.1 Preliminaries

NaVILA Backbone. NaVILA [10] is a navigation-specific Vision-Language-Action (VLA) model built upon the VILA-8B architecture [32], trained via Supervised Fine-Tuning (SFT) on large-scale navigation datasets. To incorporate historical context, the model uniformly samples N frames with a resolution of 384×384 between the initial and current frames, encoding them into 27×27 vision tokens for the LLM [16]. These inputs are processed through 32 layers, each with 32 self-attention heads, allowing the model to simultaneously attend to all past frames and instruction tokens. Based on this unified context, the model autoregressively generates mid-level action instructions (e.g., “Turn left 30 degrees”), which are executed by a low-level controller.

Distinct Characteristics of Attention Heads. The Transformer architecture, serving as the backbone of Large Language Models (LLMs), relies heavily on Multi-Head Self-Attention (MHSA) mechanisms. Although theoretically designed to perform parallel computations upon initialization, attention heads have been shown to spontaneously diverge during training and acquire distinct functional roles. Starting with the observation that specific heads attend to unique linguistic features such as syntactic relationships [11], subsequent research has demonstrated that only a small subset of these specialized heads contribute to the heavy lifting [53], allowing the majority to be pruned without significant performance degradation [40]. This specialization is now recognized as a fundamental property of LLMs [60] and has been shown to extend to LVLMs, where specific heads capture the semantic alignment between visual tokens and text [25]. Building on these insights, we analyze attention distributions to identify and leverage heads optimized for navigation tasks.

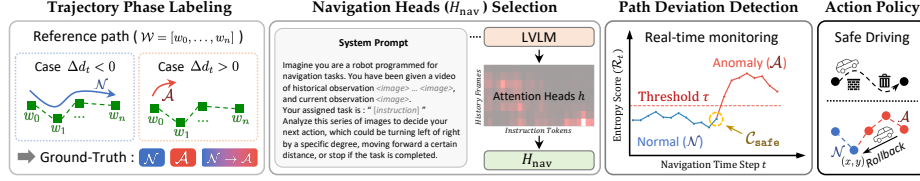


Fig. 3: Our core approach consists of four consecutive stages: (i) **Phase Labeling:** Formulating ground-truth labels based on path deviation. (ii) **Head Selection:** Identifying a subset of Navigation heads H_{nav} sensitive to these state transitions. (iii) **Anomaly Detection:** Monitoring the entropy ($\mathcal{R}_t$) of H_{nav} to detect real-time failures and preserve safe checkpoints ($\mathcal{C}_{\text{safe}}$). (iv) **Action Policy:** Deploying an RL policy for collision-free navigation and rollbacks triggered by the detected anomalies.

3.2 Trajectory Phase Categorization

To rigorously evaluate attention heads and establish a baseline for anomaly detection, we first construct a dataset by categorizing the agent’s trajectories into Normal ($\mathcal{N}$) and Anomaly ($\mathcal{A}$) phases. Given a reference path $\mathcal{W} = [w_0, \dots, w_n]$ representing a sequence of navigable waypoints for each VLN-CE episode, the navigation state evaluation is conducted through a three-step process:

(i) *Target Tracking:* At each step t , we update the target waypoint w_t^* to the closest forward waypoint along the reference path. To prevent the target from regressing, we strictly enforce a non-decreasing index constraint ($j_t \geq j_{t-1}$).

(ii) *Progress Evaluation:* We evaluate the agent’s progress by calculating the distance delta Δd_t . If the agent reaches a new waypoint, we set $\Delta d_t = 0$. If the target remains unchanged, Δd_t is the change in distance to the current target ($d_t - d_{t-1}$, where d_t is the Euclidean distance to w_t^*). Consequently, a positive Δd_t intuitively indicates a path deviation.

(iii) *Phase Classification:* We employ a one-way state machine with patience p . Starting in the $\mathcal{N}$ phase, if p consecutive translational steps yield $\Delta d_t > 0$, the state irreversibly transitions to $\mathcal{A}$, applying labels retroactively from the first deviation step. Rotational actions are excluded in $\mathcal{N}$ but included in $\mathcal{A}$ as continued disorientation. Crucially, if p consecutive on-track steps occur during $\mathcal{A}$, the episode is truncated to exclude noisy corrections.

Consequently, episodes are categorized into only normal ($\mathcal{N}$), only anomaly ($\mathcal{A}$), and normal to anomaly ($\mathcal{N} \rightarrow \mathcal{A}$), as shown in Fig. 3.

3.3 Navigation Heads Selection

Leveraging the distinct functional roles of LVLm attention heads, we identify a navigation-critical subset (H_{nav}). This subset is defined by two core criteria: (i) explicit spatio-temporal alignment, and (ii) high sensitivity to the cognitive anomalies defined in Sec. 3.2.

Spatio-Temporal Alignment. We hypothesize that specific attention heads h capture the temporal causality between the visual sequence of T history frames $\mathcal{V} = \{F_k\}_{k=0}^{T-1}$ (F_0 is the oldest observation, F_{T-1} the current frame) and the sequential navigation instruction. To isolate these heads, we extract

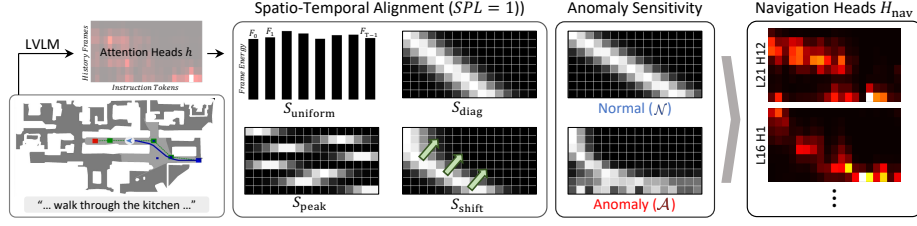


Fig. 4: Training-free spatiotemporal grounding framework for navigation. To select robust navigation heads, candidates are first evaluated using the alignment score $I_{\text{diag}}(h)$, which comprises even frame energy (S_{uniform}), focused instruction attention (S_{peak}), diagonal alignment (S_{diag}), and smooth transition (S_{shift}), and subsequently assessed for their sensitivity to attention changes between normal and anomalous navigation.

the Instruction-to-Frame attention weights to form a 2D alignment matrix $A \in \mathbb{R}^{T \times N}$ for N instruction tokens. We score these heads using a novel Spatio-Temporal alignment score $I_{\text{diag}}(h)$ that evaluates observation-level grounding and dynamic spatiotemporal shifts based on base attention dynamics.

First, observation-level cross-modal grounding is validated via frame energy uniformity (S_{uniform}) and focus sharpness (S_{peak}). To ensure that the head continuously grounds visual observations without ignoring any frame in the temporal sequence, S_{uniform} penalizes vanishing attention mass by measuring the ratio of each frame’s energy to the peak episode energy. Concurrently, to verify that the agent precisely targets a specific instruction segment rather than dispersing its focus, S_{peak} ensures a sharp, unimodal peak. Given the attention weight $A_{k,j}$ from frame k to token j , we formulate the frame energy $E_k = \sum_{j=0}^{N-1} A_{k,j}$, the normalized distribution $P_{k,j} = A_{k,j}/E_k$, and the temporal center of mass $c_k = \sum_{j=0}^{N-1} j \cdot P_{k,j}$. Defining σ_k and m_k as the standard deviation and local window mass of P_k around c_k , respectively, we compute:

$$S_{\text{uniform}} = \frac{1}{T} \sum_{k=0}^{T-1} \frac{E_k}{\max_{k'} E_{k'}}, \quad S_{\text{peak}} = \frac{1}{T} \sum_{k=0}^{T-1} \frac{1}{2} \left(\left(1 - \frac{\sigma_k}{\sigma_{\max}} \right) + m_k \right), \quad (1)$$

where $k' \in \{0, \dots, T-1\}$ indexes the frames for maximum energy, and $\sigma_{\max}$ represents the standard deviation of a uniform distribution over instruction length.

Second, to evaluate the dynamic spatiotemporal shifting trajectory, we define the ideal attention peak i_k^* for each frame, which forms a diagonal across the 2D matrix A . Accordingly, we compute the spatial proximity to this ideal diagonal (S_{diag}) and penalize erratic jumps to enforce a smooth shift towards the latter part of the instruction sequence as the navigation progresses (S_{shift}):

$$S_{\text{diag}} = 1 - \frac{1}{T} \sum_{k=0}^{T-1} \frac{|c_k - i_k^*|}{N-1}, \quad \text{where } i_k^* = \left(\frac{k}{T-1} \right) \cdot (N-1),$$

$$S_{\text{shift}} = \frac{1}{2(T-1)} \sum_{k=1}^{T-1} \left(\mathbb{I}(\Delta c_k > 0) + \exp \left(-\frac{1}{2} \left(\frac{\Delta c_k}{\Delta i_k^*} - 1 \right)^2 \right) \right), \quad (2)$$

where $\Delta c_k = c_k - c_{k-1}$ and $\Delta i_k^* = i_k^* - i_{k-1}^*$, with $\mathbb{I}(\cdot)$ as the indicator function.

Ultimately, to establish a robust baseline, the alignment score $I_{\text{diag}}(h)$ is calculated as the expectation over $\mathcal{E}_{\text{ideal}}$, a set of optimal episodes achieving a perfect Success weighted by Path Length (SPL) [2] of 1.0:

$$I_{\text{diag}}(h) = \mathbb{E}_{\tau \sim \mathcal{E}_{\text{ideal}}} \left[S_{\text{uniform}}^{(h)} \cdot S_{\text{peak}}^{(h)} \cdot \left(\lambda S_{\text{diag}}^{(h)} + (1 - \lambda) S_{\text{shift}}^{(h)} \right) \right], \quad (3)$$

where λ balances spatial precision with temporal smoothness. Heads consistently yielding high $I_{\text{diag}}(h)$ are selected as initial candidates for H_{nav} .

Cognitive Anomaly Sensitivity. While the alignment score I_{diag} identifies heads optimized for instruction following, a robust navigation head must also exhibit distinct attentional shifts during failure modes. Therefore, we evaluate the sensitivity of candidate heads by analyzing the divergence in their focus sharpness (S_{peak}) distributions between the normal ($\mathcal{N}$) and anomalous ($\mathcal{A}$) phases. This degree of separation for each head h is quantified using Cohen’s d [30]:

$$d(h) = \frac{|\mu_{\mathcal{N}} - \mu_{\mathcal{A}}|}{\sigma_{\text{pooled}}}, \quad \text{where } \sigma_{\text{pooled}} = \sqrt{\frac{(n_{\mathcal{N}} - 1)\sigma_{\mathcal{N}}^2 + (n_{\mathcal{A}} - 1)\sigma_{\mathcal{A}}^2}{n_{\mathcal{N}} + n_{\mathcal{A}} - 2}} \quad (4)$$

Here, μ and σ represent the mean and standard deviation of the focus sharpness within each partition, with n denoting the sample count. A higher $d(h)$ signifies a substantial shift or collapse in attention focus when the agent becomes stuck. Consequently, we rank the candidate heads by $d(h)$ to form H_{nav} .

3.4 Attention-Based Path Deviation Detection

To detect path deviations, we compute the mean attention entropy E_t at each time step t using the top- K heads from H_{nav} . We then monitor the relative entropy score $\mathcal{R}_t$, as the ratio of E_t to its rolling window average of size W :

$$\mathcal{R}_t = \frac{E_t}{\frac{1}{W} \sum_{i=1}^W E_{t-i} + \epsilon}, \quad \text{where } E_t = \frac{1}{|H_{\text{nav}}|} \sum_{h \in H_{\text{nav}}} \mathcal{H}(A_t^{(h)}). \quad (5)$$

Here, $\mathcal{H}(\cdot)$ denotes the normalized Shannon entropy averaged over visual frames, ensuring scale invariance. A high $\mathcal{R}_t$ indicates a sudden dispersion of attention relative to the agent’s recent history, serving as a strong precursor to navigation failure. Mirroring the patience mechanism of the ground-truth labeling, an anomaly state ($\mathcal{A}$) is registered only if $\mathcal{R}_t$ exceeds a natural threshold τ for P consecutive steps. Throughout the normal phase ($\mathcal{N}$), we continuously update a safe checkpoint $\mathcal{C}_{\text{safe}} = \{s_t, \mathcal{V}_t, \mathcal{M}_t, A_t\}$, preserving the agent’s pose $s_t = (x, y, z, \theta)$, visual history $\mathcal{V}_t$, the entropy buffer $\mathcal{M}_t = \{E_{t-W}, \dots, E_{t-1}\}$, and the attention state A_t required for future recovery. The hyperparameters (K, W, τ, P) are jointly optimized via a grid sweep on the training split, as detailed in Sec. 4.

3.5 Collision-Free Low Level Action Policy

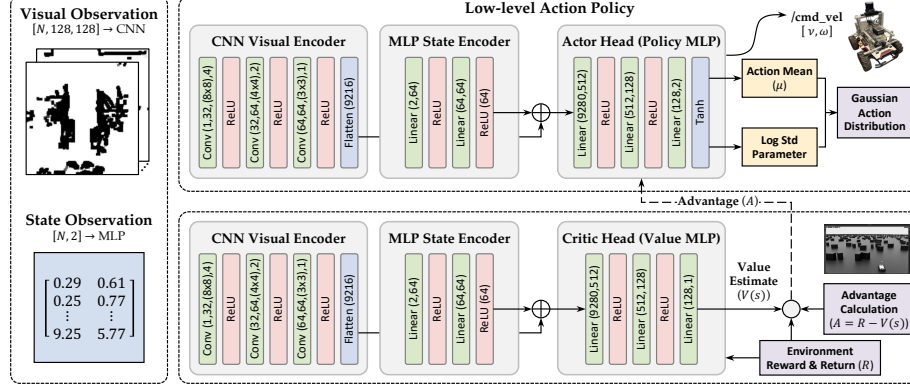


Fig. 5: The network processes 128×128 visual costmaps through a CNN encoder to extract spatial obstacle features, and relative local subgoal states (x, y) via an MLP encoder for directional guidance. These features are fused to feed two specialized heads: the **Actor head**, which outputs Gaussian distributions for velocity control commands (v, ω) , and the **Critic head**, which estimates the state-value $V(s)$. This hierarchical design enables the agent to learn reactive obstacle avoidance and robust subgoal navigation through advantage-based reinforcement learning.

To achieve safe collision avoidance and low-level control, we mirror a customized real-world robotic platform and its sensor suite within the physics-based simulator IsaacLab [42]. The agent is trained using Proximal Policy Optimization (PPO) [47] with relative goal coordinates and LiDAR data. To ensure the policy remains agnostic to specific LiDAR hardware configurations, we employ a 2D occupancy grid costmap as an intermediate observation space rather than using raw point clouds. Specifically, we apply ego-motion compensation to spatially align the map with the robot’s motion, and we couple it with a temporal decay mechanism that gradually fades historical obstacle data. This formulation enables the system to robustly handle dynamic obstacles and mitigate the impact of temporary sensor blind spots. To process these inputs, we design a lightweight and efficient actor-critic architecture (as shown in Fig. 5) that decouples the observation streams. A CNN extracts spatial features from the costmap for obstacle avoidance, while an MLP encodes the relative direction and distance to the goal for path tracking. These feature representations are subsequently fused to predict control actions. Furthermore, to promote generalization, the policy is trained in a large-scale $250\text{m} \times 200\text{m}$ virtual environment populated with 2,000 procedural obstacles (cylinders and boxes), with a minimum traversable clearance of 3.0m strictly enforced. We formulate a 14-term dense reward to jointly optimize goal-reaching, safety, and smoothness. A curriculum learning strategy progressively extends the goal distance from 5m to 20m, ensuring reliable long-horizon evasive maneuvers by consistently accumulating goal-reaching rewards. Finally, training is heavily accelerated via CUDA batching. Detailed hyperparameters are provided in the Supplementary Material.

3.6 Sim-to-Real Transfer Environment.

To bridge the sim-to-real gap, our integrated ROS [36] architecture supports both the VLA model and the RL policy. Since robust robot localization is crucial—as the VLA relies on it to save safe points for anomaly recovery, and the RL requires precise odometry for costmap-based ego-motion compensation—we adopted Fast-LIVO2 [59], a high-performance yet CPU-based odometry framework, to avoid computational bottlenecks. Fusing this with AprilTag [43] detections via an Extended Kalman Filter (EKF) establishes a reliable global coordinate system. This offloading strategy ensures limited GPU resources are reserved exclusively for the computationally intensive VLA, maximizing inference efficiency. Regarding locomotion, raw velocity commands from the RL policy often exhibit high-frequency oscillations inducing physical instability. We address this using a kinematic action smoother with a low-pass filter, acceleration clipping, and deadzone masking. Enforcing a strictly bounded acceleration profile mitigates mechanical strain and motor overload while fully preserving reactive obstacle avoidance. Building upon this robust architecture, all real-robot deployments succeeded using the exact frozen anomaly detection parameters optimized exclusively on the R2R training split (as mentioned in Sec. 4.2), strictly without environment-specific re-identification. Comprehensive results and implementation details are presented in Fig. 9 and the Supplementary Material.

4 Experiments

4.1 Experimental Setup

Simulation and Training. Experiments and RL training were conducted on a single NVIDIA RTX 6000 Ada GPU. We adopt the default NaVILA [10] configuration (384×384 resolution) as our primary baseline. To demonstrate model generalizability, we evaluate on NaVID [58]. The corresponding results and Isaac Lab implementation details are provided in the Supplementary Material.

Real-world Deployment. The framework was deployed on an AgileX Scout 2.0 mobile robot equipped with an NVIDIA Jetson AGX Orin (64 GB), a ZED 2i camera, and an RS-LiDAR. Remote control is managed via a host PC over 5G, following the setup in [20]. Detailed communication protocols and calibration parameters are available in the Supplementary Material.

4.2 Evaluating Navigation Heads for Anomaly Detection

Validation of Ground Truth Labels. As summarized in Table 1, we rigorously validate our behavioral Ground Truth (GT) labels to ensure the reliability of the anomaly detection framework. For this validation, we utilize the NaVILA [10] backbone to extract trajectories from 1,000 random episodes in the Train split, as well as all episodes in the Val-Seen and Val-Unseen splits of the VLN-CE R2R dataset [3]. Each episode is categorized into mutually exclusive

Table 1: Ground Truth (GT) analysis of trajectory phases (Sec. 3.2). On NaVILA [10] trajectories in VLN-CE R2R [3], high Cohen’s d (> 1.2) confirms strong statistical separation in waypoint distances between $\mathcal{N}$ and $\mathcal{A}$ steps. Notably, the Val-Unseen ‘Only $\mathcal{N}$ ’ ratio (56.0 %) matches the baseline Success Rate, verifying GT reliability.

Split	Episode Category				Path Distance & Effect Size		
	Total	Only $\mathcal{N}$	Only $\mathcal{A}$	$\mathcal{N} \rightarrow \mathcal{A}$	$\mathcal{N}$ Median	$\mathcal{A}$ Median	Cohen’s d
Train	1,000	617 (61.7 %)	79 (7.9 %)	304 (30.4 %)	0.13 m	2.38 m	1.54
Val-Seen	778	435 (55.9 %)	91 (11.7 %)	252 (32.4 %)	0.14 m	2.97 m	1.60
Val-Unseen	1,839	1,029 (56.0 %)	263 (14.3 %)	547 (29.7 %)	0.16 m	2.53 m	1.27

classes. Step-level path-distance analysis reveals that, even without explicit spatial thresholds, our state-machine-based labeling successfully captures distinct navigational behaviors. Specifically, $\mathcal{N}$ steps tightly adhere to the optimal path with median deviations of 0.13–0.16 m, while $\mathcal{A}$ steps diverge significantly to 2.38–2.97 m. The resulting large effect size (Cohen’s $d > 1.2$) statistically confirms that our behavior-driven criteria robustly isolate severe deviations from minor steering noise. Furthermore, the only $\mathcal{N}$ ratio in the Val-Unseen split (56.0 %) matches the baseline’s terminal Success Rate, justifying the macroscopic validity of our GT. Finally, analyzing the temporal structure of $\mathcal{N} \rightarrow \mathcal{A}$ transitions reveals a median $\mathcal{N}$ phase of 6–7 steps followed by an $\mathcal{A}$ phase of 9–10 steps. This balanced phase distribution confirms that the anomaly-onset positions are assigned, providing sufficient baseline history to characterize the normal navigational context and an adequate temporal window for detection.

Path Deviation Detection Performance. To verify the robustness and generalizability of our mathematically formalized, data-driven framework, we derive optimal configurations strictly from the R2R train split and evaluate them zero-shot on validation splits. As shown in Fig. 6a, we extract a subset of H_{nav} and conduct an extensive grid search over 9,000 combinations, comprising the ensemble size ($K \in \{1, \dots, 10\}$), patience ($P \in \{1, \dots, 10\}$), sliding window ($W \in \{1, \dots, 10\}$), and 9 values for the activation threshold (τ). Optimization results on the train split (Fig. 6) indicate that, under a 10 % False Episode

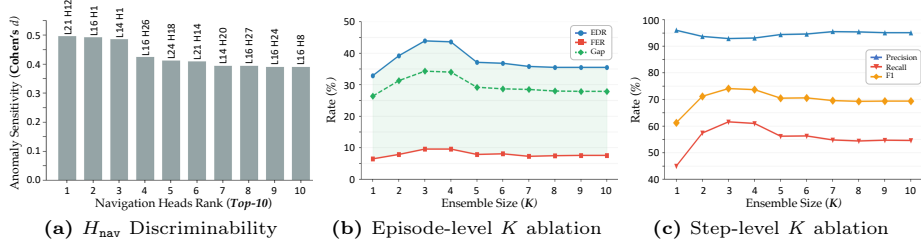


Fig. 6: Ablation study of navigation heads (H_{nav}) and ensemble size (K) on the R2R train split. (a) Discriminability of attention heads measured by Cohen’s d ; higher values indicate greater sensitivity to anomalous behaviors. (b) Episode-level detection performance across K . Thresholds are optimized via grid search to maximize Episode Detection Rate (EDR) while constraining False Episode Rate (FER) within 5–10 % (Gap = EDR – FER). (c) Step-level classification performance on $\mathcal{N} \rightarrow \mathcal{A}$ episodes. Notably, an ensemble of just $K = 3$ heads achieves 43.9 % EDR and 9.6 % FER at the episode level, alongside 92.9 % Precision, 61.6 % Recall, and 74.1 % F1 at the step level.

Table 2: Performance comparison of training-free anomaly detection on R2R validation splits. Evaluating H_{nav} and hyperparameters derived solely from the train split, we baseline against rule-based heuristics [1, 39] to ensure a fair comparison with near-zero inference overhead. Our approach significantly outperforms these baselines in both episode-level net advantage (Gap) and step-level F1. Notably, while heuristics suffer from severe recall bottlenecks (missing $>60\%$ of anomalies), our method robustly captures navigation failures without requiring additional trainable modules.

Method	R2R Val-Seen						R2R Val-Unseen					
	All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level			All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level		
	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$
Stagnation [39]	24.5 %	4.8 %	19.7 %	99.7 %	31.3 %	47.7 %	29.6 %	5.9 %	23.7 %	95.6 %	38.7 %	55.1 %
Act.Failure [1]	0.3 %	6.0 %	-5.7 %	70.6 %	2.8 %	5.5 %	1.5 %	6.3 %	-4.8 %	77.9 %	5.6 %	10.4 %
Uncertainty	1.5 %	0.0 %	1.5 %	100.0 %	0.8 %	1.6 %	1.7 %	0.0 %	1.7 %	100.0 %	2.1 %	4.1 %
Ours	44.6 %	11.7 %	32.9 %	91.3 %	68.6 %	78.3 %	41.9 %	9.6 %	32.2 %	92.5 %	65.1 %	76.4 %

Rate (FER) constraint, the ensemble combination of $K = 3$ achieves the peak Episode Detection Rate (EDR). At the step level, this configuration effectively captures anomalies with 92.9 % Precision (avoiding false alarms), 61.6 % Recall (identifying path deviations), and a 74.1 % F1 score (overall balance).

Table 2 compares our approach against established zero-inference-overhead heuristics [1, 39] and an uncertainty-based approach on the validation splits, strictly utilizing the optimized heads and hyperparameters derived from the train data. Our method significantly outperforms these rigid baselines across both episode- and step-level metrics. Notably, Position Stagnation [39] (relying on fixed (x, y) displacement thresholds), Action Failure [1] (monitoring coordinate shifts strictly during forward actions), and the uncertainty baseline (triggering when action token probability falls below 50 %) all universally suffer from critically low recall. Because LVLMS remain severely overconfident during hallucinations and persistently execute movement commands, these approaches leave anomalies largely undetected (resulting in low recall), which is the primary challenge our framework is designed to overcome. In contrast, monitoring semantic misalignment between instructions and visual observations robustly captures these failures. Despite a marginal compromise on false alarms (FER), our method achieves

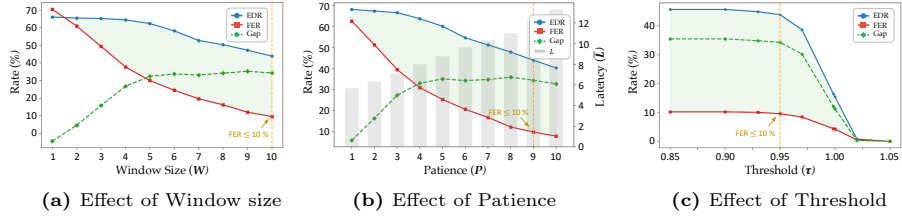
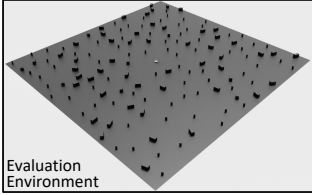


Fig. 7: Hyperparameter grid search for the proposed anomaly detector ($K = 3$) on the R2R train split. (a) Both EDR and detection Gap (EDR $-$ FER) increase monotonically, saturating around $W = 7$. (b) Increasing P effectively suppresses FER, albeit at the cost of a linearly increasing mean detection latency. (c) The detector exhibits sharp sensitivity near $\tau = 1.0$, where both EDR and FER drop rapidly. Ultimately, driven by the FER $< 10\%$ constraint, we select ($W = 10, P = 9, \tau = 0.95$) that achieves an EDR of 43.9 %, thereby preventing unnecessary interruptions during normal navigation.

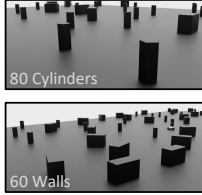
higher recall across the Val-Seen (68.6 %) and Val-Unseen (65.1 %) splits, yielding superior F1 scores (78.3 % and 76.4 %, respectively). This confirms our approach provides a far more generalizable signal than coordinate- or logit-based mechanisms. Qualitative results and successful validation on an additional VLA model [58] are detailed in Fig. 9 and the Supplementary Material.

4.3 RL Obstacle Avoidance Performance

Metric	SR $\uparrow$	CR $\downarrow$	TR $\downarrow$
APF [26]	47.4 %	8.6 %	43.9 %
DWA [14]	17.8 %	79.6 %	2.6 %
MPPI [57]	2.5 %	91.2 %	6.3 %
TEB [46]	47.2 %	43.6 %	9.2 %
Ours	87.2 %	5.3 %	7.5 %



Evaluation Environment



80 Cylinders
60 Walls

(a) RL Policy Performance
(b) Evaluation Environments

Fig. 8: Evaluation of the low-level RL obstacle avoidance policy. (a) Quantitative comparison of average performance metrics (SR, CR, and TR) against classical local planning baselines [14, 26, 46, 57]. Our proposed policy significantly outperforms traditional methods, achieving the highest average success rate of 87.2 %. (b) Visualization of the IsaacLab evaluation environment. The testbed comprises a 60 m $\times$ 60 m subterrain populated with procedurally generated cylindrical obstacles and wall-shaped boxes to comprehensively assess collision-free navigation.

Evaluation Setup. We evaluate our RL obstacle avoidance policy in IsaacLab [41]. The testbed comprises a single 60 m $\times$ 60 m subterrain (Fig. 8b) populated with 80 cylindrical obstacles and 60 wall-shaped boxes (all 2.0 m high). A 2.0 m minimum clearance ensures feasible paths. Across four target-distance scenarios (5 m to 20 m), 512 independent episodes are evaluated per distance scenario, yielding 2,048 total episodes. The policy maps local costmaps to continuous velocity commands bounded to $v \in [-0.5, 0.5]$ m/s and $\omega \in [-1.0, 1.0]$ rad/s. Episodes conclude upon: (i) reaching within 0.5 m of the target (Success Rate, SR); (ii) stalling > 3.0 s or exceeding 150.0 s (Terminate Rate, TR); or (iii) colliding with an obstacle exceeding 200 N of force (Collision Rate, CR).

Evaluation Analysis. Table 8a compares our policy against four classical local planners using purely local observations (a 128 $\times$ 128 costmap and relative goal position). Among the baselines, APF [26] attains a relatively high SR (47.4 %) but suffers a severe TR (43.9 %) due to frequent entrapment in local minima. TEB [46] shows a comparable SR (47.2 %) but struggles with collisions (CR 43.6 %) in dense configurations. DWA [14] and MPPI [57] fail entirely (CR > 79 %), as their short-horizon trajectory sampling cannot anticipate complex obstacle layouts. In contrast, our RL policy achieves the highest SR (87.2 %), outperforming the best baseline by nearly 40 percentage points, while maintaining a low CR (5.3 %) and TR (7.5 %). This demonstrates that our tailored reward function and extensive environmental interaction effectively equip the policy with implicit spatial reasoning, allowing it to smoothly navigate dense layouts where classical reactive planners crash or stall. Detailed per-distance metrics are provided in the Supplementary Material.

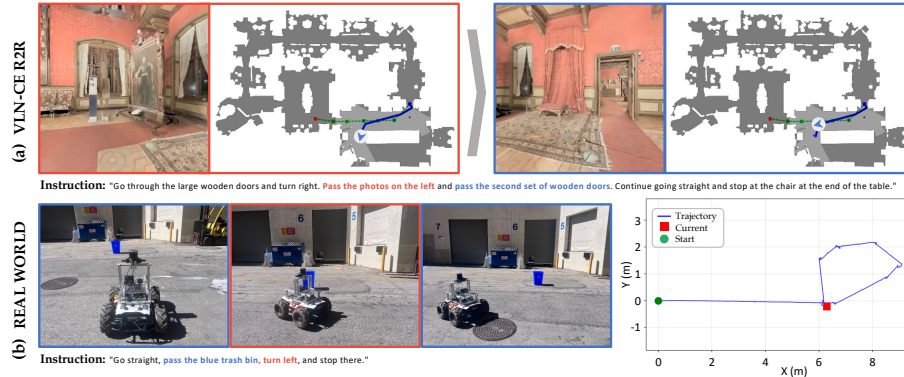


Fig. 9: (a) Hallucination-induced path deviation and recovery in VLN-CE simulation. (b) Demonstration of VLA-RL integrated path recovery on a custom on-device robot.

4.4 Real-world Evaluation

As shown in Fig. 9, we evaluated our proposed framework on a custom VLA-driven robot. Upon detecting path deviations, the RL policy safely executes a trajectory rollback. Notably, these real-world deployments were conducted using the exact frozen parameters optimized exclusively on the R2R train split, strictly without any environment-specific re-identification. Setup details and further experimental results are provided in the Supplementary Material.

5 Conclusion

We present a training-free anomaly detection and recovery framework for VLA models. By analyzing the internals of a frozen VLA, we identified a sparse set of Navigation Heads (H_{nav}) that naturally capture the spatiotemporal alignment between visual observations and instructions. Building on this, we propose a real-time system detecting path deviations by monitoring attention dynamics without computational overhead. For safe physical execution, we integrated this signal with a low-level RL policy for reactive obstacle avoidance and direct safe recovery. Evaluations results demonstrate our approach effectively mitigates navigation failures caused by VLA hallucinations. Ultimately, this research provides a practical solution for deploying reliable, self-monitoring embodied agents.

Limitations and Future Work. While our proposed identification of navigation heads enables cost-free detection, simply resetting the robot’s physical position without internal resolution can trigger repetitive cognitive errors. Implementing active replanning or re-prompting to align these internal states would break such error loops, ultimately enhancing overall navigation success rates. Furthermore, within the context of prior research on pruning redundant attention heads for model compression [40], our identification of navigation heads offers a strategic pathway for targeted optimization. By selectively retaining only the task-essential heads, inference speeds could be significantly accelerated, thereby enabling more efficient and responsive operation in on-device deployments.

Acknowledgements

We acknowledge financial support from the National Institute of Food & Agriculture of the U.S. Department of Agriculture (Grant No. 2024-67021-42528) and the NVIDIA Academic Grants Program. This work was also supported by the Culture, Sports, and Tourism R&D Program through a Korea Creative Content Agency grant funded by the Ministry of Culture, Sports and Tourism in 2024 (International Collaborative Research and Global Talent Development for the Development of Copyright Management and Protection Technologies for Generative AI, No. RS-2024-00345025), as well as an Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No. RS-2019-III190079, Artificial Intelligence Graduate School Program, Korea University).

References

1. An, D., Wang, H., Wang, W., Wang, Z., Huang, Y., He, K., Wang, L.: Etpnav: Evolving topological planning for vision-language navigation in continuous environments. *IEEE TPAMI* (2024) [2](#), [4](#), [12](#)
2. Anderson, P., Chang, A., Chaplot, D.S., Dosovitskiy, A., Gupta, S., Koltun, V., Kosecka, J., Malik, J., Mottaghi, R., Savva, M., et al.: On evaluation of embodied navigation agents. *arXiv preprint arXiv:1807.06757* (2018) [8](#)
3. Anderson, P., Wu, Q., Teney, D., Bruce, J., Johnson, M., Sünderhauf, N., Reid, I., Gould, S., Van Den Hengel, A.: Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In: *CVPR*. pp. 3674–3683 (2018) [1](#), [10](#), [11](#)
4. Beemelmans, T.: ros2_calib: A Multi-Sensor Calibration Tool for ROS2. https://github.com/ika-rwth-aachen/ros2_calib (2025). <https://doi.org/10.5281/zenodo.17119720>, accessed: 2026-03-11 [16](#)
5. Chen, K., Chen, J.K., Chuang, J., Vázquez, M., Savarese, S.: Topological planning with transformers for vision-and-language navigation. In: *CVPR*. pp. 11276–11286 (2021) [2](#)
6. Chen, L., Zhao, H., Liu, T., Bai, S., Lin, J., Zhou, C., Chang, B.: An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In: *ECCV*. pp. 19–35 (2024) [3](#)
7. Chen, S., Guhur, P.L., Schmid, C., Laptev, I.: History aware multimodal transformer for vision-and-language navigation. In: *NeurIPS*. vol. 34, pp. 5834–5847 (2021) [1](#)
8. Chen, S., Guhur, P.L., Tapaswi, M., Schmid, C., Laptev, I.: Think global, act local: Dual-scale graph transformer for vision-and-language navigation. In: *CVPR*. pp. 16537–16547 (2022) [1](#)
9. Chen, Z., Zhao, Z., Luo, H., Yao, H., Li, B., Zhou, J.: HALC: Object hallucination reduction via adaptive focal-contrast decoding. *arXiv preprint arXiv:2403.00425* (2024) [2](#)
10. Cheng, A.C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., et al.: Navila: Legged robot vision-language-action model for navigation. In: *RSS* (2025) [1](#), [3](#), [5](#), [10](#), [11](#)
11. Clark, K., Khandelwal, U., Levy, O., Manning, C.D.: What does BERT look at? an analysis of BERT’s attention. In: *BlackboxNLP*. pp. 276–286 (2019) [5](#)

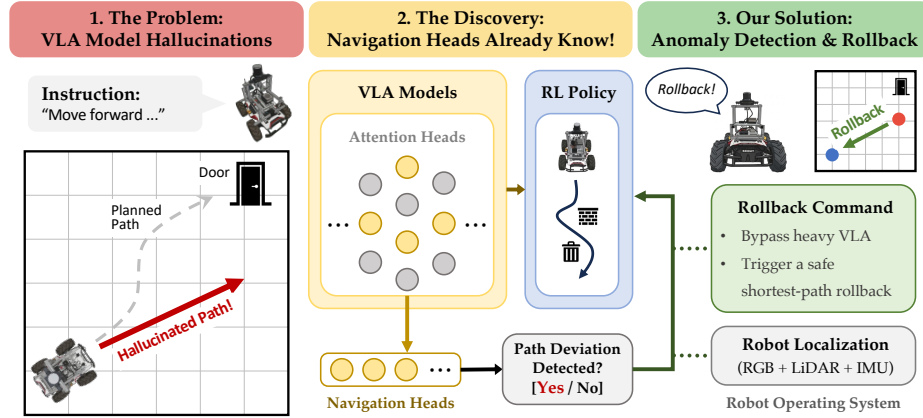
12. Du, Y., Fu, T., Chen, Z., Li, B., Su, S., Zhao, Z., Wang, C.: VL-Nav: Real-time vision-language navigation with spatial reasoning. *arXiv preprint arXiv:2502.00931* (2025) [3](#)
13. Elnoor, M., Weerakoon, K., Seneviratne, G., Xian, R., Guan, T., Jaffar, M.K.M., Rajagopal, V., Manocha, D.: Robot navigation using physically grounded vision-language models in outdoor environments. *arXiv preprint arXiv:2409.20445* (2024) [3](#)
14. Fox, D., Burgard, W., Thrun, S.: The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine* **4**(1), 23–33 (2002) [4](#), [13](#), [11](#)
15. Fried, D., Hu, R., Cirik, V., Rohrbach, A., Andreas, J., et al.: Speaker-follower models for vision-and-language navigation. In: *NeurIPS* (2018) [1](#), [4](#)
16. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al.: The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024) [5](#)
17. Hong, Y., Wu, Q., Qi, Y., Rodriguez-Opazo, C., Gould, S.: Vln bert: A recurrent vision-and-language bert for navigation. In: *CVPR*. pp. 1643–1653 (2021) [1](#)
18. Hornung, R., Urbanek, H., Klodmann, J., Osendorfer, C., van der Smagt, P.: Model-free robot anomaly detection. In: *IROS*. pp. 3676–3683 (2014) [2](#), [4](#)
19. Hu, Y., Lin, F., Zhang, T., Yi, L., Gao, Y.: Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842* (2023) [2](#), [4](#)
20. Jeong, J., Vu, T.A., Jony, M., Ahmad, S., Rahman, M.M., Kim, S., Jawed, M.K.: Agrichrono: A multi-modal dataset capturing crop growth and lighting variability with a field robot. *arXiv preprint arXiv:2508.18694* (2025) [10](#)
21. Jeong, J., Vu, T.A., Lahoti, R., Wang, J., Alumootil, V., Kim, S., Jawed, M.K.: Vision-guided targeted grasping and vibration for robotic pollination in controlled environments. *arXiv preprint arXiv:2510.06146* (2025) [16](#)
22. Ji, T., Sivakumar, A.N., Chowdhary, G., Driggs-Campbell, K.: Proactive anomaly detection for robot navigation with multi-sensor fusion. *IEEE RA-L* **7**(2), 4975–4982 (2022) [2](#), [4](#)
23. Kahn, G., Abbeel, P., Levine, S.: BADGR: An autonomous self-supervised learning-based navigation system. *IEEE RA-L* **6**(2), 1312–1319 (2021) [4](#)
24. Kahn, G., Abbeel, P., Levine, S.: LaND: Learning to navigate from disengagements. *IEEE RA-L* **6**(2), 1872–1879 (2021) [4](#)
25. Kang, S., Kim, J., Kim, J., Hwang, S.J.: Your large vision-language model only needs a few attention heads for visual grounding. In: *CVPR*. pp. 9339–9350 (2025) [5](#)
26. Khatib, O.: Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research* **5**(1), 90–98 (1986) [13](#), [11](#)
27. Krantz, J., Lee, S.: Sim-2-sim transfer for vision-and-language navigation in continuous environments. In: *ECCV*. pp. 588–603 (2022) [3](#)
28. Krantz, J., Wijmans, E., Majumdar, A., Batra, D., Lee, S.: Beyond the nav-graph: Vision-and-language navigation in continuous environments. In: *ECCV*. pp. 104–120 (2020) [1](#)
29. Ku, A., Anderson, P., Patel, R., Ie, E., Baldridge, J.: Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In: *EMNLP*. pp. 4392–4412 (2020) [2](#), [3](#)
30. Lakens, D.: Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for *t*-tests and ANOVAs. *Frontiers in Psychology* **4**, 863 (2013) [8](#)

31. Lin, F., Nai, R., Hu, Y., You, J., Zhao, J., Gao, Y.: OneTwoVLA: A unified vision-language-action model with adaptive reasoning. arXiv preprint arXiv:2505.11917 (2025) [4](#)
32. Lin, J., Yin, H., Ping, W., Lu, Y., Molchanov, P., Tao, A., Mao, H., Kautz, J., Shoeybi, M., Han, S.: Vila: On pre-training for visual language models. In: CVPR (2024) [5](#)
33. Liu, Z., Zhu, L., Shi, B., Zhang, Z., Lou, Y., Yang, S., et al.: Nvila: Efficient frontier visual language models. arXiv preprint arXiv:2412.04468 (2024) [3](#)
34. Ma, C.Y., Lu, J., Wu, Z., AlRegib, G., Kira, Z., Socher, R., Xiong, C.: Self-monitoring navigation agent via auxiliary progress estimation. arXiv preprint arXiv:1901.03035 (2019) [2](#), [4](#)
35. Ma, C.Y., Wu, Z., AlRegib, G., Xiong, C., Kira, Z.: The regretful agent: Heuristic-aided navigation through progress estimation. In: CVPR. pp. 6732–6740 (2019) [2](#)
36. Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W.: Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics* **7**(66), eabm6074 (2022) [10](#)
37. Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., Shroff, G.: LSTM-based encoder-decoder for multi-sensor anomaly detection. arXiv preprint arXiv:1607.00148 (2016) [4](#)
38. Marafioti, A., Zohar, O., Farré, M., Noyan, M., Bakouch, E., Cuenca, P., Zakka, C., Allal, L.B., Lozhkov, A., Tazi, N., et al.: SmolVLM: Redefining small and efficient multimodal models. arXiv preprint arXiv:2504.05299 (2025) [1](#)
39. Marder-Eppstein, E., Berger, E., Foote, T., Gerkey, B., Konolige, K.: The office marathon: Robust navigation in an indoor office environment. In: ICRA. pp. 300–307 (2010) [4](#), [12](#)
40. Michel, P., Levy, O., Neubig, G.: Are sixteen heads really better than one? In: NeurIPS (2019) [5](#), [14](#)
41. Mittal, M., Roth, P., Tigue, J., Richard, A., Zhang, O., Du, P., Serrano-Munoz, A., Yao, X., Zurbrugg, R., Rudin, N., et al.: Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. arXiv preprint arXiv:2511.04831 (2025) [13](#)
42. Mittal, M., Roth, P., Tigue, J., Richard, A., Zhang, O., Du, P., et al.: Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. arXiv preprint arXiv:2511.04831 (2025) [9](#), [4](#), [5](#)
43. Olson, E.: AprilTag: A robust and flexible visual fiducial system. In: ICRA. pp. 3400–3407 (2011) [10](#), [14](#)
44. Park, D., Hoshi, Y., Kemp, C.C.: A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder. *IEEE RA-L* **3**(3), 1544–1551 (2018) [4](#)
45. Rajvanshi, A., Sikka, K., Lin, X., Lee, B., Chiu, H.P., Velasquez, A.: SayNav: Grounding large language models for dynamic planning to navigation in new environments. In: ICAPS. vol. 34, pp. 464–474 (2024) [2](#), [4](#)
46. Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F., Bertram, T.: Trajectory modification considering dynamic constraints of autonomous robots. In: ROBOTIK 2012; 7th German Conference on Robotics. pp. 1–6 (2012) [13](#), [11](#)
47. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017) [9](#)
48. Serrano-Muñoz, A., Chrysostomou, D., Bøgh, S., Arana-Arexolaleiba, N.: skrl: Modular and flexible library for reinforcement learning. *Journal of Machine Learn-*

- ing Research **24**(254), 1–9 (2023), <http://jmlr.org/papers/v24/23-0112.html> **9, 10**
49. Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., et al.: Smolvla: A vision-language-action model for affordable and efficient robotics. arXiv preprint arXiv:2506.01844 (2025) **3**
 50. Taioli, F., Rosa, S., Castellini, A., Natale, L., Del Bue, A., Farinelli, A., Cristani, M., Wang, Y.: Mind the error! detection and localization of instruction errors in vision-and-language navigation. In: IROS. pp. 12993–13000 (2024) **2, 4**
 51. Tan, H., Yu, L., Bansal, M.: Learning to navigate unseen environments: Back translation with environmental dropout. In: NAACL. pp. 2610–2621 (2019) **1**
 52. Tang, F., Liu, C., Xu, Z., Hu, M., Huang, Z., Xue, H., Chen, Z., Peng, Z., Yang, Z., Zhou, S., et al.: Seeing far and clearly: Mitigating hallucinations in MLLMs with attention causal decoding. In: CVPR. pp. 26147–26159 (2025) **2**
 53. Voita, E., Talbot, D., Moiseev, F., Sennrich, R., Titov, I.: Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In: ACL. pp. 5797–5808 (2019) **5**
 54. Wang, H., Liang, W., Van Gool, L., Wang, W.: Dreamwalker: Mental planning for continuous vision-language navigation. In: ICCV. pp. 10873–10883 (2023) **2**
 55. Wang, Z., Li, X., Yang, J., Liu, Y., Hu, J., Jiang, M., Jiang, S.: Lookahead exploration with neural radiance representation for continuous vision-language navigation. In: CVPR. pp. 13753–13762 (2024) **1**
 56. Wellhausen, L., Ranftl, R., Hutter, M.: Safe robot navigation via multi-modal anomaly detection. IEEE RA-L **5**(2), 1326–1333 (2020) **2, 4**
 57. Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J.M., Boots, B., Theodorou, E.A.: Information theoretic MPC for model-based reinforcement learning. In: ICRA. pp. 1714–1721 (2017) **13, 11**
 58. Zhang, J., Wang, K., Xu, R., Zhou, G., Hong, Y., Fang, X., Wu, Q., Zhang, Z., Wang, H.: Navid: Video-based vlm plans the next step for vision-and-language navigation. arXiv preprint arXiv:2402.15852 (2024) **1, 3, 10, 13**
 59. Zheng, C., Xu, W., Zou, Z., Hua, T., Yuan, C., He, D., Zhou, B., Liu, Z., Lin, J., Zhu, F., et al.: FAST-LIVO2: Fast, direct LiDAR–inertial–visual odometry. IEEE Transactions on Robotics **41**, 326–346 (2024) **10, 14**
 60. Zheng, Z., Wang, Y., Huang, Y., Song, S., Yang, M., Tang, B., Xiong, F., Li, Z.: Attention heads of large language models: A survey. arXiv preprint arXiv:2409.03752 (2024) **5**
 61. Zhu, Y., Weng, Y., Zhu, F., Liang, X., Ye, Q., Lu, Y., Jiao, J.: Self-motivated communication agent for real-world vision-dialog navigation. In: ICCV. pp. 1594–1603 (2021) **4**

Your Vision-Language-Action Model Already Has Attention Heads For Path Deviation Detection

Supplementary Material



Contents

A	Expanding Path Deviation Detection	2
A.1	Computational Overhead Analysis	2
A.2	Additional Experiments on the RxR Dataset	2
A.3	Additional Experiments on the NaVid Model	3
B	Obstacle Avoidance RL Action Policy	4
B.1	IsaacLab Simulation Setup	4
B.2	Dynamic 2D Costmap Observation	6
B.3	Reinforcement Learning Configuration	8
B.4	RL Policy Computational Resources	10
B.5	Performance Metrics by Target Distance	11
B.6	Real-World Reactive Navigation of the RL Policy	11
C	Real-Robot Platform System Implementation	14
C.1	System Specifications	14
C.2	Kinematic Action Smoother	17
C.3	State Estimation Comparison	18
D	Real-World Deployment Results	20

A Expanding Path Deviation Detection

A.1 Computational Overhead Analysis

Table 3: Computational overhead analysis on a single NVIDIA RTX 6000 Ada GPU. The integration of our path deviation detection module introduces a marginal latency increase of less than 20 ms per step, with no additional GPU memory required.

Configuration	Total Time	Alloc VRAM	Peak VRAM
NaVILA (Baseline)	583.5 ms	17,145.6 MB	18,730.3 MB
NaVILA + Ours	603.3 ms	17,145.6 MB	18,730.3 MB
Difference	+19.8 ms	0.0 MB	0.0 MB

Expanding upon the performance discussions in Sec. 4.2 of the main paper, Table 3 details the precise computational requirements of our path deviation detection mechanism on an NVIDIA RTX 6000 Ada GPU. The table demonstrates that our module introduces a near-zero computational overhead, adding less than 20 ms of latency per step without requiring additional peak GPU memory. Additionally, the computational overhead on the NVIDIA Jetson AGX Orin edge device can be found in Table 14. During inference, the computational logic remains highly streamlined. Provided that the task-relevant navigation heads (H_{nav}) and the optimal natural threshold (τ) have been pre-identified through offline statistical analysis on the training data, online real-time inference merely requires a simple $\mathcal{O}(1)$ lookup. This allows the system to instantly extract the attention distributions specific to H_{nav} . The subsequent anomaly detection logic comprises calculating the Shannon entropy over the visual tokens, computing the relative difference against a rolling-window mean, and comparing it against the predefined threshold. Because these steps consist entirely of lightweight scalar operations, the entire detection mechanism operates without introducing any new neural network computations beyond what the base model already performs.

A.2 Additional Experiments on the RxR Dataset

Following the primary results presented in Table 2 of the main paper, we conducted additional evaluations on the RxR (Room-across-Room) dataset [29] to demonstrate the dataset generalizability and robustness of our approach. To identify the navigation heads (H_{nav}), we strictly followed the exact same offline analysis protocol, deriving them exclusively from the RxR training set to prevent any potential data leakage. Once identified, these head configurations and their corresponding hyperparameters were completely frozen and evaluated on both the Val-Seen and Val-Unseen splits, with the results presented in Table 4. Considering that the RxR dataset is characterized by significantly higher linguistic complexity and longer, more intricate spatial trajectories compared to R2R, these results sufficiently prove the data-level robustness of our methodology. In particular, successfully maintaining an F1 score over 50 % across both the Val-Seen and Val-Unseen splits confirms that our attention-based deviation detection serves as a robust, dataset-agnostic indicator of navigational confidence.

Table 4: Path Deviation Detection Performance on the RxR Dataset. Extending the primary evaluation on the R2R dataset (Table 2 of the main paper), we further validate our proposed method on the RxR dataset [29]. The detection mechanism, configured with H_{nav} and hyperparameters derived exclusively from the training set, demonstrates robust generalizability across RxR validation splits. Most importantly, it successfully maintains an overall F1 score of over 50 % in both environments by balancing exceptionally high Precision with robust Recall.

Method	RxR Val-Seen						RxR Val-Unseen					
	All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level			All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level		
	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$
Ours	36.1 %	6.7 %	29.4 %	96.9 %	39.8 %	56.5 %	32.7 %	9.7 %	23.0 %	93.2 %	34.4 %	50.3 %

A.3 Additional Experiments on the NaVid Model

To rigorously verify the architecture-agnostic generalizability of our proposed framework, we extended our evaluation to NaVid [58]. Despite having fundamentally different backbones compared to NaVILA [10]—specifically, employing Vicuna-7B and EVA-CLIP rather than VILA-8B and CLIP—we strictly applied the protocol from Sec. 4.2 of the main paper without any heuristic modifications. During this validation, the specific head combinations and hyperparameters derived exclusively from the R2R Train split remained completely frozen. As shown in Table 5, the empirical results exhibit a similar trend to Table 2 in the main paper. While conventional heuristics still suffer from low failure detection rates, our methodology successfully identified attention heads exhibiting analogous spatiotemporal alignment patterns within NaVid. Consequently, our approach achieved superior Recalls of 57.3 % and 53.2 %, alongside the highest overall F1 scores of 66.7 % and 63.2 % in the Val-Seen and Val-Unseen environments, respectively. These findings strongly confirm that our framework operates independently of specific VLA parameters, successfully translating internal model interpretability into a highly robust robot control signal.

Table 5: Extending our evaluation beyond the primary NaVILA backbone, we validate our training-free anomaly detection pipeline on NaVid [58]. Configured with heads and hyperparameters optimized strictly on the R2R Train split, our method demonstrates robust architecture generalizability. Most importantly, it significantly outperforms rigid heuristics and uncertainty baselines, achieving the highest Recall and overall F1 scores.

Baseline (Navid ¹)	R2R Val-Seen						R2R Val-Unseen					
	All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level			All Episodes			$\mathcal{N} \rightarrow \mathcal{A}$ Step-level		
Method	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	EDR $\uparrow$	FER $\downarrow$	Gap $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$
Stagnation	22.4 %	6.3 %	16.1 %	<u>91.7 %</u>	<u>30.6 %</u>	<u>45.9 %</u>	18.1 %	7.0 %	<u>11.1 %</u>	<u>95.6 %</u>	<u>23.9 %</u>	<u>38.2 %</u>
Act.Failure	3.1 %	<u>6.8 %</u>	-3.7 %	80.6 %	3.3 %	6.3 %	2.6 %	<u>8.8 %</u>	-6.2 %	70.5 %	3.7 %	7.1 %
Uncertainty	37.0 %	31.1 %	5.9 %	100.0 %	17.3 %	29.5 %	28.9 %	27.1 %	1.8 %	96.1 %	18.5 %	31.0 %
Ours	<u>32.3 %</u>	8.1 %	24.2 %	79.7 %	57.3 %	66.7 %	<u>27.9 %</u>	12.1 %	15.8 %	77.8 %	53.2 %	63.2 %

B Obstacle Avoidance RL Action Policy

B.1 IsaacLab Simulation Setup

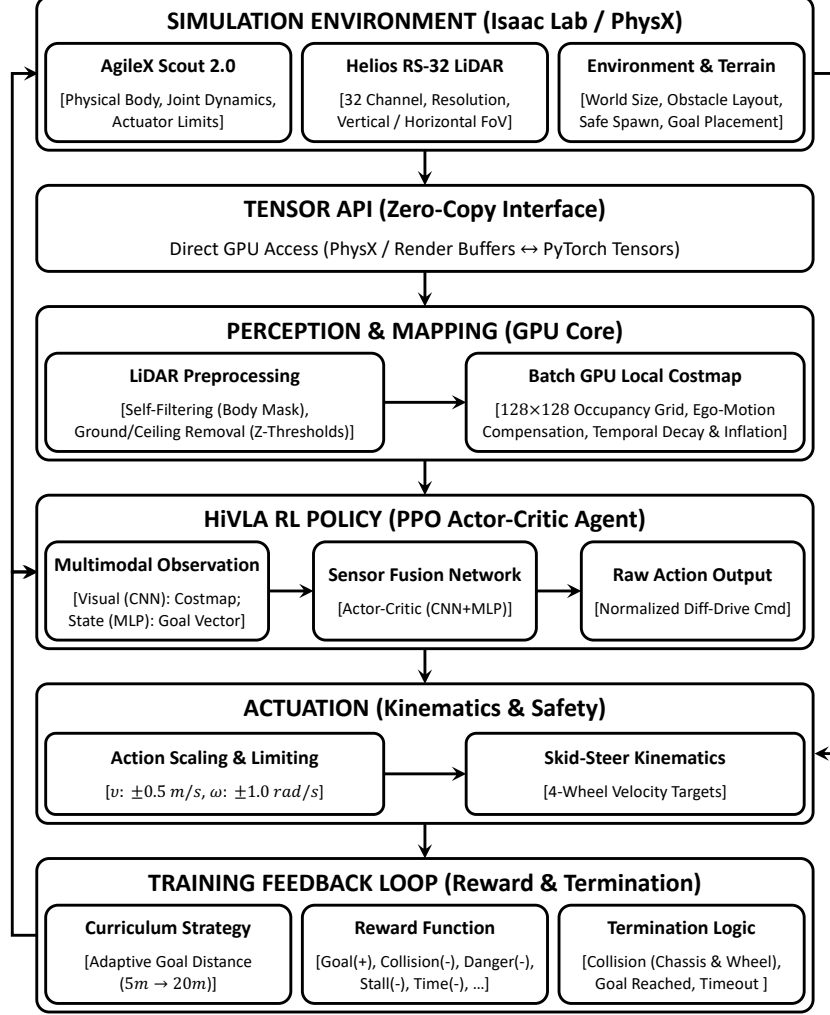


Fig. 10: RL training diagram. Utilizing IsaacLab [42] and PhysX, we construct a simulation environment that precisely aligns with real-world dynamics, incorporating the USD-defined robot geometry and sensor extrinsics while tuning skid-steer friction, damping, and actuator forces alongside the 32-channel LiDAR configurations. To facilitate reward discovery and ensure stable convergence, we employ a curriculum strategy that adaptively scales the navigation challenge by increasing the goal distance from 5 m to 20 m across 20 sub-terrains mixed with cylinders and walls, as illustrated in Fig. 9 of the main paper. With rewards tailored for collision avoidance and goal-reaching, this large-scale training of 1,024 agents on a single NVIDIA RTX 6000 Ada GPU enables comprehensive exploration of the state space and robust policy optimization.

Table 6: Configuration of the IsaacLab [42] Simulation Environment

Category	Parameter	Value
Robot & Kinematics	Wheel Radius (R) / Wheelbase (L)	0.165 m / 0.582 m
	Robot Dimensions (Length $\times$ Width)	1.06 m $\times$ 0.90 m
	Max Linear / Angular Velocity	± 0.5 m/s / ± 1.0 rad/s
	Policy Control Frequency	15 Hz (≈ 0.067 s/step)
3D LiDAR Sensor	Sensor Mounting Height	0.63 m
	Channels / Max Range	32 / 10.0 m
	Vertical / Horizontal FOV	-55° to $+15^\circ$ / 360°
	Horizontal Resolution	0.2° (57,632 rays/scan)
Terrain & Environment	Sub-terrains (Size / Count)	60 m $\times$ 60 m / 20 tiles
	Cylinder Obstacles	80 per tile (Radius: 0.25 m, Height: 2.0 m)
	Wall Obstacles	60 per tile (1.5 m $\times$ 0.75 m $\times$ 2.0 m)
	Min Obstacle Clearance	1.8 m
	Min Spawn Clearance	1.5 m
Termination Conditions	Goal Reached Threshold	0.5 m
	Timeout Limit	150.0 s
	Collision Threshold (Chassis / Wheels)	> 200.0 N
	Stall Condition	< 0.2 m displacement in 3.0 s

To facilitate robust sim-to-real transfer, we meticulously designed the RL training environment within IsaacLab [42], as outlined in Fig. 10. Crucially, this simulation is engineered to precisely mirror our real-world physical setup, as demonstrated in Fig. 13. The comprehensive list of physical and environmental parameters utilized for this configuration is detailed in Table 6.

Robot Configuration and Kinematics. To minimize the reality gap, we constructed a custom USD model of our mobile robot and imported it into the simulation at a 1:1 scale. Because the physical robot operates on a skid-steer differential drive mechanism, we explicitly engineered the left and right wheel velocities to be governed by the kinematic equations $v_{left} = (v - \omega \cdot L/2)/R$ and $v_{right} = (v + \omega \cdot L/2)/R$, where v and ω denote the linear and angular velocities, respectively. We strictly applied the real-world specifications: a wheelbase of $L = 0.582$ m and a wheel radius of $R = 0.165$ m. The RL policy utilizes a tanh activation to output continuous actions bounded within $[-1, 1]$, which we explicitly mapped to a maximum linear velocity of 0.5 m/s and a maximum angular velocity of 1.0 rad/s. To define the physical collision boundaries during navigation, we adopted a conservative rectangular robot dimension of 1.06 m $\times$ 0.90 m, incorporating an empirical safety margin over the bare chassis measured directly on the physical robot. To balance rapid reactive obstacle avoidance with computational efficiency while maintaining physical realism, we designed the policy to operate at a control frequency of 15 Hz (i.e., ≈ 0.067 s per step), allowing a maximum translational displacement of ≈ 0.033 m per action step, while the underlying physics engine simulates dynamics at 60 Hz.

Sensor Setup. To guarantee seamless sim-to-real transfer of perception, we configured the 3D LiDAR module provided by IsaacLab to faithfully replicate the mounting position and hardware specifications of the physical Helios RS-32 LiDAR used in our real-world deployments. Specifically, the simulated sensor is mounted at a height of 0.63 m above the ground. It uses 32 vertical channels to enforce a vertical field of view (FOV) from -55° to $+15^\circ$, along with a full 360° horizontal FOV. With a 0.2° horizontal resolution, the sensor casts 57,632 rays per scan up to a maximum range of 10.0 m, providing a dense geometric representation of the surroundings identical to the real robot’s sensory input.

Environment and MDP Definitions. Training occurs across a procedurally generated terrain composed of 20 sub-terrains. To enforce a reproducible and physically viable evaluation standard, we populated each sub-terrain tile with 80 cylinders (radius 0.25 m, height 2.0 m) and 60 wall-shaped boxes ($1.5\text{ m} \times 0.75\text{ m} \times 2.0\text{ m}$). We configured a rejection-sampling algorithm to position the obstacles, ensuring a minimum obstacle-to-obstacle clearance of 1.8 m, with a 3.0 m-wide obstacle-free platform preserved at the center of each tile. At episode initialization, we programmed the robot to spawn 0.5 m above the ground, with a randomized yaw orientation to prevent physics clipping, and to enforce a strict 1.5 m obstacle-free spawn clearance. We defined the Markov Decision Process (MDP) to terminate under four distinct conditions: (i) reaching within 0.5 m of the goal, (ii) exceeding the maximum time limit of 150.0 s, (iii) experiencing physical collisions exceeding 200.0 N on either the chassis or the wheels (after a brief 20-step warm-up period to ignore initial spawn impacts), or (iv) stalling (accumulating less than 0.2 m of total displacement for over 3.0 s).

B.2 Dynamic 2D Costmap Observation

Rather than relying exclusively on sparse 1D LiDAR arrays, our RL policy operates on a dynamic 2D occupancy grid costmap. This intermediate representation, as illustrated in Fig. 11, equips the agent with spatial memory without requiring Recurrent Neural Networks (RNNs) and ensures sim-to-real robustness through sensor agnosticism. While raw 1D data is easily overfitted to specific hardware specifications, the 2D costmap decouples the input space from the raw sensor stream, allowing the policy to remain fully compatible with different LiDAR models or depth cameras without retraining. The visual observation space is a single-channel 128×128 tensor that captures a $6.4\text{ m} \times 6.4\text{ m}$ physical area centered on the robot, yielding a dense, uniform 0.05 m/pixel resolution.

Hardware-Accelerated Pipeline and Sim-to-Real Alignment. Processing 57,632 LiDAR rays on the CPU for thousands of parallel agents in simulation induces a severe computational bottleneck. To resolve this, we constructed the entire mapping pipeline using CUDA-optimized PyTorch operations and executed it directly on the GPU in FP16 precision. To eliminate the sim-to-real gap in the perception module, this identical PyTorch codebase is deployed on the physical robot’s NVIDIA Jetson AGX Orin. By setting the batch size to 1, the physical robot leverages its native Tensor Cores to process the costmap in real time with the same logic, completely bypassing CPU-GPU transfer overheads.

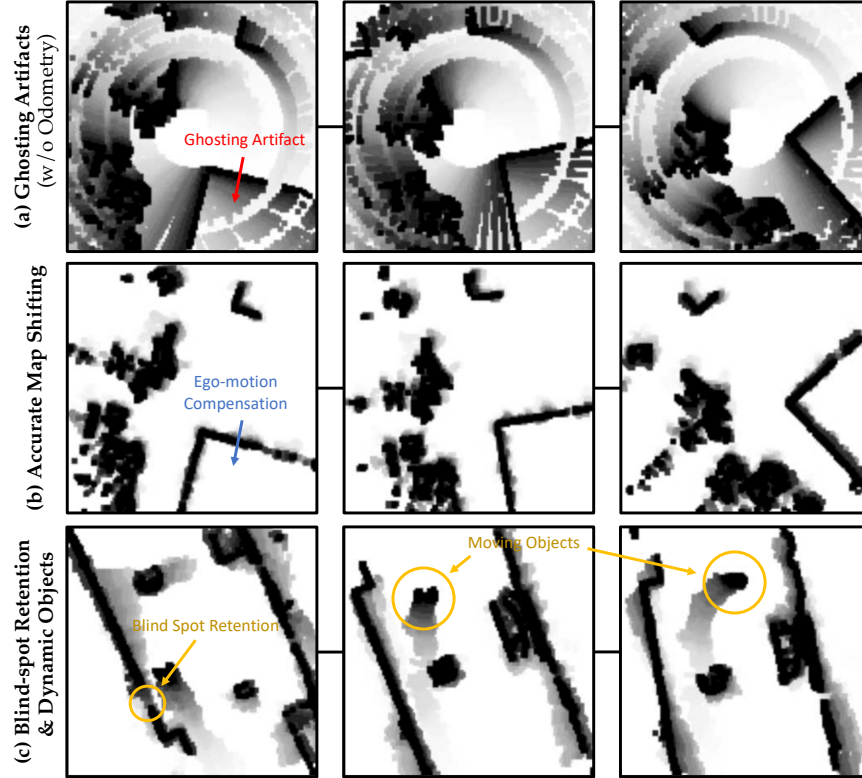


Fig. 11: Dynamic 2D Costmap Observation with Ego-Motion Compensation and Temporal Decay. (a) Without proper odometry, uncompensated ego-motion causes severe ghosting artifacts during a clockwise rotation, permanently corrupting the spatial memory. (b) By incorporating strict ego-motion compensation during the same rotation, the local costmap accurately shifts in accordance with the robot’s movement, maintaining a clean and consistent representation. (c) Applying an optimal temporal decay factor ($\gamma = 0.9$) allows the system to temporarily retain static obstacles even when they enter the sensor’s blind spot, while gracefully fading the historical trails of dynamic moving objects.

Ego-Motion Compensation via Zero-Allocation Warping. To maintain a globally consistent map, the local costmap must accurately shift in response to the robot’s movement. Our pipeline integrates 3D LiDAR point clouds with high-frequency odometry data to compute the relative translational and rotational displacements at each policy step. To eliminate overhead during real-time execution, we pre-allocate an affine transformation matrix buffer, fundamentally eradicating memory allocation and Garbage Collection (GC) spikes during navigation. Subsequently, we warp the previous costmap using nearest-neighbor grid sampling (`F.grid_sample`) to apply the inverse of the robot’s motion. During this process, newly revealed unknown areas entering the field of view are strictly zero-padded to ensure safety.

Temporal Decay for Dynamic Environments. Real-world environments inherently contain moving objects. If the costmap remains static (i.e., a decay factor of 1.0), a passing pedestrian would leave a permanent obstacle trail, completely blocking the robot’s path. Conversely, decaying the map too rapidly causes the agent to instantly forget static obstacles in temporary blind spots, leading to collisions. To resolve this dilemma, we apply a temporal decay factor of $\gamma = 0.9$ at each step. This specific rate strikes an optimal balance for spatial memory—safely remembering temporary blind spots while gracefully fading the historical trails of dynamic entities.

Spatial Filtering and Safety Margins. Finally, to ensure the policy reacts exclusively to genuine navigational hazards, we filter the LiDAR points along the Z-axis, retaining only data within $0.1 \text{ m} \leq Z \leq 1.0 \text{ m}$ to mask out ground noise and overhead structures. Furthermore, we apply a geometric self-masking filter to prevent the robot from perceiving its own chassis or antennas as obstacles. Rather than relying on simple CAD dimensions, the boundaries of this $1.10 \text{ m} \times 0.80 \text{ m}$ exclusion footprint were empirically derived by directly measuring the physical RS-LiDAR sensor’s self-occlusion view. Any points falling within this measured zone are strictly ignored. To prevent the policy from shaving corners too closely, we apply a 2D Max Pooling operation (3×3 kernel) over the grid, geometrically inflating all valid obstacles by an exact 1-pixel (0.05 m) safety margin with negligible computational overhead.

B.3 Reinforcement Learning Configuration

We utilize Proximal Policy Optimization (PPO) to train the navigation policy within the IsaacLab environment. The training configuration is structurally divided into the design of a dense reward function and the specification of PPO network hyperparameters, as detailed in Tables 7 and 8, respectively.

Reward Function. To cultivate robust, human-like navigation in highly constrained environments, we engineered a dense reward function (r_{total}) structured into four domains (Table 7). To formalize the robot’s spatial perception, we define d_{corridor} as the minimum obstacle distance within a forward safety corridor, flagging the path as blocked ($\mathbb{I}_{\text{blk}} = 1$) if $d_{\text{corridor}} < 0.8 \text{ m}$. We calculate the lateral tightness ($\tau_{\text{side}} = \max(\tau_L, \tau_R) \in [0, 1]$) to quantify the narrowness of the immediate surroundings. The rationale for each component is as follows: (i) **Task Progression:** Relying solely on a sparse success bonus (r_{success}) granted at a 0.5 m threshold is inefficient for long-horizon navigation. We employ a potential-based shaping reward (r_{goal}) utilizing an exponential function of the 2D goal distance (d_t). This provides a steepening, continuous gradient, effectively preventing the agent from being trapped in local minima. A step-wise time penalty (r_{time}) further encourages efficient routing. (ii) **Kinematic Control:** To ensure dynamically feasible trajectories, the projected velocity reward (r_{vel}) incentivizes forward motion (v_x) aligned with the goal ($\cos(\psi_{\text{err}})$). Crucially, scaling this reward by τ_{side} encourages the policy to accelerate aggressively in open spaces while decelerating cautiously in narrow aisles. This is complemented by continu-

Table 7: Dense Reward Function Configuration. The total reward is the sum of these 12 components. $\mathbb{I}_{\text{blk}}$ denotes the path-blocked indicator ($d_{\text{corridor}} < 0.8$ m).

Category	Component	Formulation & Condition
Task	Goal Progress (r_{goal})	$10 \cdot (d_{t-1} - d_t) + 50 \cdot (e^{-0.25d_t} - e^{-0.25d_{t-1}})$
	Goal Success (r_{success})	+20.0 if $d_t < 0.5$ m
	Time Penalty (r_{time})	-0.05 per step
Kinematics	Projected Velocity (r_{vel})	$3.0 \cdot v_x \cos(\psi_{\text{err}}) \cdot (1 - 0.85 \tau_{\text{side}}) \cdot (1 - \mathbb{I}_{\text{blk}})$
	Heading Alignment (r_{head})	$1.0 \cdot (1 - \psi_{\text{err}} /\pi) \cdot \mathbb{I}(v_x > 0.1) \cdot (1 - \mathbb{I}_{\text{blk}})$
	Yaw Error Reduction (r_{yaw})	$2.0 \cdot (\psi_{\text{err}} _{t-1} - \psi_{\text{err}} _t)$
	Micro-Displacement (r_{move})	$0.2 \cdot \tanh(\Delta s/0.03)$
Safety	Collision Penalty (r_{col})	-50.0 if force > 200 N (chassis or wheels)
	Proximity Danger (r_{danger})	$-3.0 \cdot (1 - \text{clamp}(d_{\text{corridor}}/2.18, 0, 1))^2$
Behavior	Angular Penalty (r_{ang})	$-1.0 \cdot \omega_z \cdot (1 - \tau_{\text{side}}) \cdot (1 - \mathbb{I}_{\text{blk}})$
	Centering Reward (r_{center})	$2.5 \cdot (\tau_R - \tau_L) \cdot \omega_z$
	Stall Penalty (r_{stall})	$\max(-0.5 \cdot \max(t_{\text{stall}} - 1.0, 0)^2, -2.0)$

ous heading alignment (r_{head}), step-wise yaw error reduction (r_{yaw}), and micro-displacement (r_{move}) rewards to maintain momentum without lateral drifting. (iii) **Safety Enforcement:** For reliable sim-to-real transfer, a heavy discrete penalty (r_{col}) is strictly applied for physical collisions exceeding 200 N of force on the chassis or wheels. To proactively deter reckless driving, we introduce a proximity danger penalty (r_{danger}) formulated as a quadratic function of d_{corridor} . This establishes a virtual repulsive field that punishes dangerous approaches long before physical collisions occur. (iv) **Behavioral Regularization:** Skid-steer robots often suffer from oscillatory freezing in tight spaces. We penalize excessive angular velocity (r_{ang}) in open areas and punish prolonged stalls (r_{stall}) exceeding a 1.0 s grace period. Most importantly, a centering reward (r_{center}) leverages the spatial imbalance between the left and right clearances ($\tau_R - \tau_L$) to naturally steer the robot toward the wider side, keeping it smoothly centered through bottlenecks.

PPO Hyperparameters. Expanding upon the reinforcement learning methodology introduced in Sec. 3.5 of the main paper, Table 8 details the SKRL-based PPO configuration [48] for stabilizing 1,024 parallel IsaacLab environments. The specific rationale for each key hyperparameter is as follows: (i) **Temporal Discounting and Advantage Estimation:** We apply the standard discount factor of $\gamma = 0.99$ to preserve long-horizon reward signals across diverse procedurally generated sub-terrains. This temporal weighting is paired with a Generalized Advantage Estimation (GAE) parameter of $\lambda = 0.95$ to strike an optimal balance between bias and variance. (ii) **Observation and Batch Processing:** We completely disabled the default state preprocessor. Since our primary visual observation is a structured 2D occupancy costmap, standard continuous normalization would distort its absolute geometric scale and distinct boundary

Table 8: PPO Network and Training Hyperparameters (SKRL [48])

Hyperparameter	Value	Hyperparameter	Value
Algorithm	PPO	Learning Rate	3×10^{-4}
Parallel Environments	1,024	LR Scheduler	KL-Adaptive (0.008)
Rollout Length	96 steps	Discount Factor (γ)	0.99
Learning Epochs	5	GAE Parameter (λ)	0.95
Mini-batches	4	Total Timesteps	100,000
State Preprocessor	Disabled	Checkpoint Interval	5,000 steps

values, thereby degrading the spatial memory of the CNN encoder. For policy optimization, collecting 96 rollout steps across all 1,024 environments yields a massive batch of 98,304 transitions per iteration. To maximize sample efficiency, these transitions are rigorously optimized over 5 epochs and 4 mini-batches. (iii) **Action Space and Actor Architecture:** Driven by the non-holonomic kinematics of our target wheeled robot, the action space is explicitly restricted to two continuous control commands: the forward linear velocity (v_x) and the angular velocity (ω_z). To ensure policy stability and prevent the generation of physically unbounded commands, the final output layer of the actor network employs a tanh activation function. This strictly bounds the raw model outputs to the normalized range $[-1, 1]$ and subsequently scales them to match the robot chassis’s maximum physical velocity limits during simulation. (iv) **Adaptive Learning Rate:** To maintain policy stability across diverse procedurally generated sub-terrains, we employ a KL-adaptive learning rate scheduler starting at a base rate of 3×10^{-4} . By dynamically scaling the learning rate based on a target KL divergence threshold of 0.008, we strictly bound the magnitude of policy updates. This mechanism prevents excessively large policy steps, which can lead to catastrophic forgetting, ensuring that the robot retains its critical collision-avoidance reflexes throughout 100,000 training timesteps.

B.4 RL Policy Computational Resources

Table 9: Computational overhead analysis of the RL Action Policy. Evaluated on a single NVIDIA RTX 6000 Ada GPU, the lightweight CNN + MLP architecture (4.89 M parameters) achieves a near-instantaneous inference latency of 0.2 ms per step with minimal VRAM utilization.

Configuration	Total Time	Alloc. VRAM	Peak VRAM
Ours (RL Policy)	0.2 ms	26.9 MB	27.9 MB

The RL Action Policy is designed as a streamlined CNN + MLP architecture with approximately 4.89 M parameters, specifically optimized for high-frequency reactive control. As detailed in Table 9, this lightweight configuration enables ultra-low-latency inference, requiring only 0.2 ms per step on an NVIDIA RTX

6000 Ada GPU. With a minimal VRAM footprint of less than 28 MB, this RL policy provides the essential agility for real-time obstacle avoidance and stability. By maintaining a negligible computational load even during continuous high-rate actuation, it ensures overall system efficiency and ultimately guarantees a stable control frequency of 10 Hz during real-world deployment, as detailed in Sec. C.

B.5 Performance Metrics by Target Distance

Table 10: Navigation performance by distance. SR: Success Rate, CR: Collision Rate, TR: Timeout Rate.

Distance	5m			10m			15m			20m		
Method	SR $\uparrow$	CR $\downarrow$	TR $\downarrow$	SR $\uparrow$	CR $\downarrow$	TR $\downarrow$	SR $\uparrow$	CR $\downarrow$	TR $\downarrow$	SR $\uparrow$	CR $\downarrow$	TR $\downarrow$
APF [26]	60.6 %	5.9 %	33.6 %	48.4 %	10.0 %	41.6 %	41.4 %	10.0 %	48.6 %	39.3 %	8.8 %	51.9 %
DWA [14]	17.6 %	75.8 %	<u>6.6 %</u>	16.2 %	82.8 %	1.0 %	21.1 %	78.1 %	0.8 %	16.2 %	81.8 %	2.0 %
MPPI [57]	3.7 %	90.0 %	6.3 %	2.7 %	91.6 %	<u>5.7 %</u>	1.6 %	91.6 %	6.8 %	2.0 %	91.4 %	<u>6.6 %</u>
TEB [46]	<u>65.6 %</u>	24.0 %	10.4 %	<u>51.0 %</u>	40.6 %	8.4 %	41.0 %	50.4 %	8.6 %	31.1 %	59.4 %	9.6 %
Ours	83.8 %	<u>6.4 %</u>	9.8 %	88.1 %	4.1 %	7.8 %	90.0 %	5.1 %	<u>4.9 %</u>	86.7 %	5.5 %	7.8 %

Expanding upon Sec. 4.3 of the main paper, Table 10 details the performance across target distances from 5 m to 20 m to analyze long-horizon robustness beyond the average metrics. As distance increases, baselines exhibit significant degradation: APF’s termination rate (TR) rises to 51.9 % at 20 m due to cumulative entrapment in local minima, while TEB’s collision rate (CR) climbs to 59.4 % in complex layouts. DWA and MPPI consistently fail across all ranges with CR exceeding 75 %. In contrast, our policy maintains a Success Rate (SR) of 86.7 % even at the maximum distance, more than doubling the performance of the strongest baseline (APF, 39.3 %). Notably, despite the fourfold increase in distance, our CR remains exceptionally stable below 6.5 %, proving that the policy effectively avoids local minima and maintains consistent safety in dense environments where classical reactive planners frequently stall or collide.

B.6 Real-World Reactive Navigation of the RL Policy

To evaluate the low-level action policy for obstacle avoidance, we demonstrated the real-world navigation performance of the standalone RL policy without VLA guidance. Given simple forward commands over distances of tens of meters, the agent operates at 10 Hz to safely navigate through dynamic and unstructured environments. As shown in Fig. 12, (a) the policy successfully avoids both static objects and moving pedestrians indoors, and (b) it demonstrates robust collision avoidance against complex, irregularly shaped trees in an outdoor orchard farm.

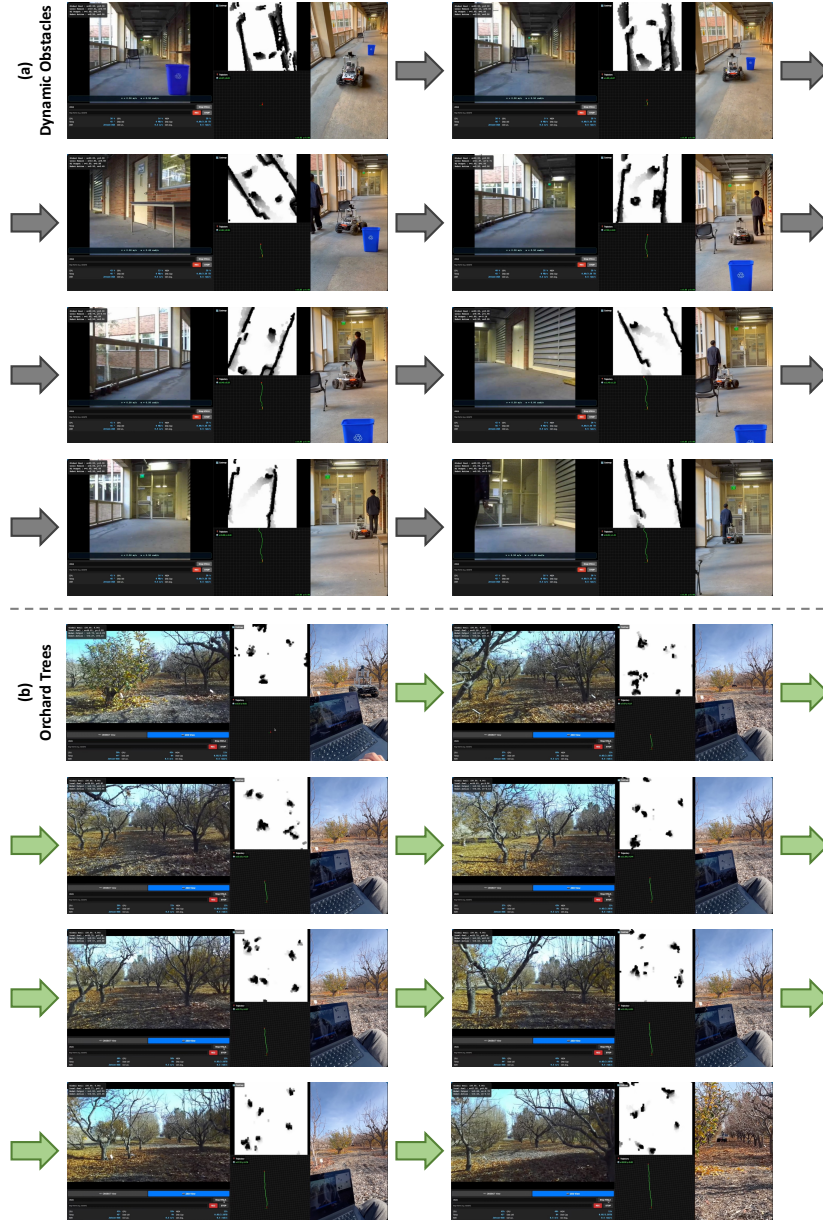


Fig. 12: Real-world navigation performance of the standalone RL policy. Operating at 10 Hz without VLA guidance, the agent safely navigates dynamic environments over tens of meters. **(a)** Indoors, it successfully avoids static objects and moving pedestrians. **(b)** In an outdoor orchard, it demonstrates robust collision avoidance against complex, irregular trees.

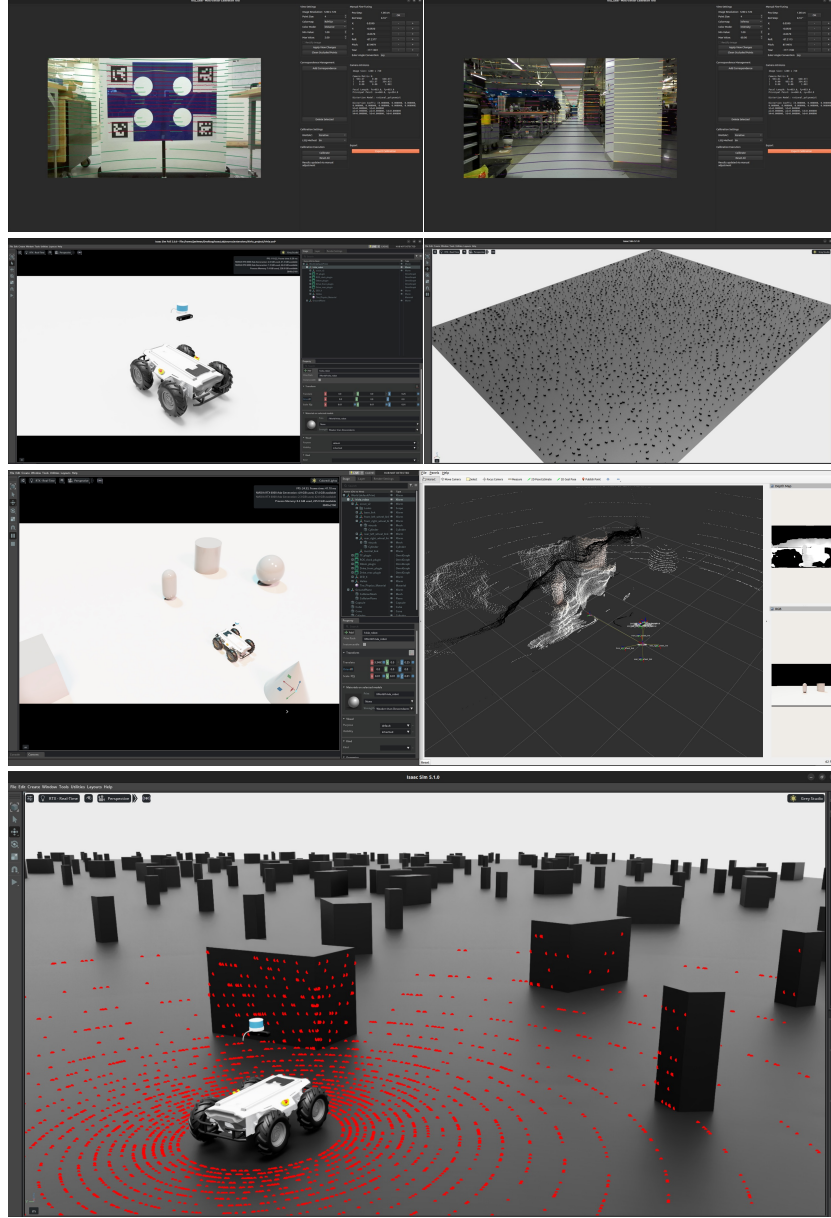


Fig. 13: Overview of the robotic system and simulation environment. **First row:** sensor calibration of the ZED 2i camera and RS-32 LiDAR. **Second row:** USD robot model with sensor extrinsics and training terrains. **Third row:** Testing real-time ROS 2 integration within Isaac Sim. **Fourth row:** training visualization of a single agent among 1,024 parallel instances.

C Real-Robot Platform System Implementation

C.1 System Specifications

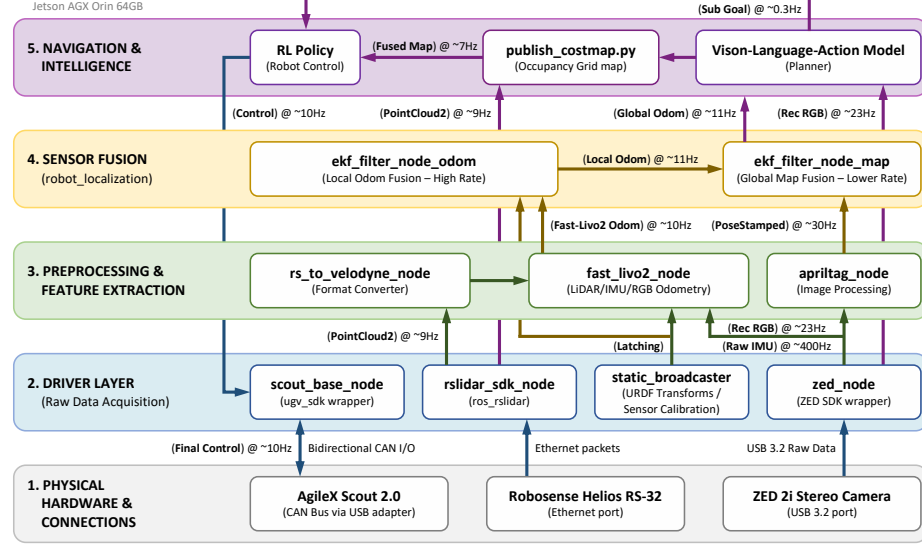


Fig. 14: ROS 2 System Architecture. The on-device pipeline integrates multi-modal data from a skid-steer robot, a 32-channel spinning LiDAR, and a stereo RGB-D camera. For precise robot localization and efficient resource allocation, we employ Fast-LIVO2 [59], which leverages LiDAR, RGB, and IMU sensor streams. This is coupled with an Extended Kalman Filter (EKF) that fuses AprilTag [43] detections to establish and maintain a robust globally fixed coordinate system. The navigation core operates asynchronously, comprising a ~ 0.3 Hz VLA planner for high-level sub-goal generation and a ~ 10 Hz RL policy that executes reactive control for obstacle avoidance.

Expanding upon Sec. 3.6 of the main paper, we developed a comprehensive hardware-software bridge on our custom skid-steer robot to successfully transfer the learned policies from simulation to the physical world. As shown in Fig. 14, this architecture is explicitly designed to balance the high-frequency demands of reactive control with the intensive computational loads of visual reasoning. Furthermore, it enables effortless teleoperation and real-time monitoring of the robot through a dedicated WebUI, as illustrated in Fig. 15.

Software Setup. To support this architecture, we configured a robust on-device environment, with detailed specifications summarized in Table 11. The core compute engine is an NVIDIA Jetson AGX Orin 64GB running Ubuntu 22.04 and ROS 2 Humble. To maximize edge computing efficiency, the deep learning stack is fully optimized using JetPack 6.2.1 and CUDA 12.6. Within this ROS 2 ecosystem, the system effectively manages the asynchronous execution of multi-rate nodes. It seamlessly handles intensive inference for the 8-billion-parameter NaVILA model and attention-based path deviation detection, while simultaneously

Table 11: System specifications for physical deployment. This table outlines the comprehensive hardware and software stack configured on the NVIDIA Jetson AGX Orin edge computer. It details the full hierarchical navigation pipeline, encompassing the 8-billion-parameter NaVILA model for high-level reasoning, the attention-based path deviation detection mechanism (parameterized by window size W and thresholds), and the lightweight 4.89 M parameter RL action policy for reactive local control.

Category	Component	Specification
Hardware	Compute	NVIDIA Jetson AGX Orin Dev. 64 GB
	Mobile Base	AgileX Scout 2.0
	Camera	ZED 2i (Stereo RGB-D)
	LiDAR	RoboSense Helios RS-32
System Software	JetPack	6.2.1 (L4T R36.4.7)
	Kernel	5.15.148-tegra
	OS	Ubuntu 22.04 (aarch64)
	CUDA	12.6
	cuDNN	9.3.0
	Python	3.10.12
ROS 2 Stack	ROS 2 Distro	Humble
NaVILA Model	Base Model	VILA (LLaMA-3 8B)
	Vision Encoder	SigLIP-So400M/patch14/384
	Checkpoint	navila-llama3-8b-8f
	Input Resolution	384×384
	Inference Frames	8 (7 history + 1 current)
Deviation Detection	Selected Heads ([L, H])	[21, 12], [16, 1], [14, 1]
	Window Size	$W = 10$ steps
	Patience Threshold	$P = 9$ steps
	Natural Threshold	$\tau = 0.95$
RL Action Policy	Architecture	CNN + MLP
	Parameters	4.89 M
	Costmap Input	$1 \times 128 \times 128$ (Occupancy Grid)
	Goal Input	1×2 ($\Delta x, \Delta y$ in local frame)

executing high-frequency Fast-LIVO2 sensor-fusion odometry and a lightweight RL action policy for reactive control on the AgileX Scout 2.0 mobile base. Furthermore, to facilitate real-time monitoring without degrading the main computational pipeline, we deployed a custom WebUI utilizing a Janus server and Flask. As shown in Fig. 15, this integration ensures low-latency multimedia streaming and seamless teleoperation for the user.

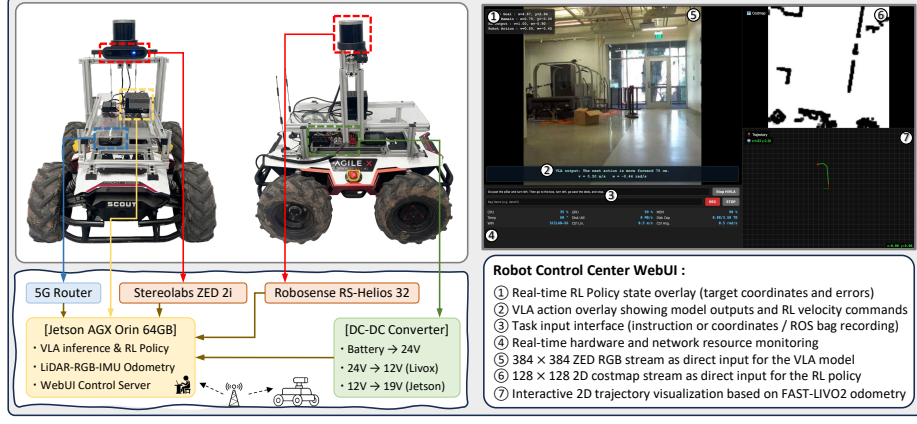


Fig. 15: Physical Robot Setup and Control Center WebUI. (Left) Hardware configuration on the AgileX Scout 2.0 mobile base. A custom enclosure rigidly collocates the Helios-RS 32 LiDAR and ZED 2i stereo camera. The onboard NVIDIA Jetson AGX Orin processes the VLA inference, RL policy, and Fast-LIVO2 odometry. Power distribution is managed via a dedicated DC-DC converter, while a 5G router ensures low-latency remote connectivity. (Right) The custom Robot Control Center WebUI was deployed for teleoperation and monitoring.

Hardware Setup. As visualized in Fig. 15, the 32-channel Helios-RS 32 LiDAR is mounted centrally at a height of 0.63m to maximize ground terrain visibility, while the ZED 2i stereo camera is positioned to provide an unobstructed egocentric view. This hardware configuration precisely aligns with the environment described in Sec. C. To rigidly secure these sensors, we fabricated a custom hardware enclosure. Furthermore, to ensure accurate spatial alignment between the sensors, we performed cross-sensor extrinsic calibration using an interactive, targetless calibration tool [4, 21]. As depicted in Fig. 13, this calibration process yields the following transformation matrix, $T_{\text{lidar}}^{\text{cam}}$:

$$T_{\text{lidar}}^{\text{cam}} = \begin{bmatrix} 0.0017 & -0.0349 & 0.9994 & 0.0537 \\ -1.0000 & 0.0017 & 0.0018 & 0.0593 \\ -0.0017 & -0.9994 & -0.0349 & -0.0938 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

This precise spatial mapping plays a crucial role in deriving highly accurate results across the entire navigation pipeline: RGB-based path planning for the VLA model, LiDAR-based reactive control for the RL policy, and robust LiDAR-RGB-IMU fusion for the Fast-LIVO2 odometry.

C.2 Kinematic Action Smoother

Table 12: Kinematic Action Smoother Configuration. Hyperparameters defining the physical constraints and symmetric acceleration limits for the skid-steer robot.

Parameter	Symbol	Value
Max Linear Velocity	$v_{\max}$	0.5 m/s
Max Angular Velocity	$\omega_{\max}$	0.5 rad/s
Linear Acceleration Limit	$a_{\max}$	1.0 m/s ²
Angular Acceleration Limit	$a_{\max}^{\omega}$	5.0 rad/s ²
Deadzone Threshold	ϵ	0.02

At each control step t operating at 10 Hz ($\Delta t = 0.1$ s), the RL policy outputs target velocity commands $(v_t^{\text{tgt}}, \omega_t^{\text{tgt}})$. Directly applying these discontinuous commands to the physical robot can induce severe mechanical oscillations and physically unsafe accelerations. To ensure stable and safe locomotion, we implemented a three-stage Kinematic Action Smoother pipeline, utilizing the physical constraints and hyperparameters detailed in Table 12.

(i) **Velocity Saturation:** The raw target velocities are first strictly bounded to the physical limits of the robot chassis to prevent hardware overload:

$$\hat{v}_t = \text{clip}(v_t^{\text{tgt}}, -v_{\max}, v_{\max}), \quad \hat{\omega}_t = \text{clip}(\omega_t^{\text{tgt}}, -\omega_{\max}, \omega_{\max}) \quad (7)$$

(ii) **Symmetric Acceleration Clamping:** We compute the instantaneous acceleration over the elapsed time Δt . To prevent jerky starts and stops, we enforce symmetric linear ($a_{\max}$) and angular ($a_{\max}^{\omega}$) acceleration limits. At the 10 Hz control frequency, this configuration allows the robot to safely decelerate from the maximum linear velocity ($v_{\max}$) to a complete stop in exactly 5 steps (0.5 s) and execute a full angular direction reversal in 2 steps (0.2 s):

$$a_t = \frac{\hat{v}_t - v_{t-1}}{\Delta t}, \quad \bar{v}_t = v_{t-1} + \text{clip}(a_t, -a_{\max}, a_{\max}) \cdot \Delta t \quad (8)$$

$$a_t^{\omega} = \frac{\hat{\omega}_t - \omega_{t-1}}{\Delta t}, \quad \bar{\omega}_t = \omega_{t-1} + \text{clip}(a_t^{\omega}, -a_{\max}^{\omega}, a_{\max}^{\omega}) \cdot \Delta t \quad (9)$$

(iii) **Deadzone Suppression:** Finally, sub-threshold commands are explicitly zeroed to suppress motor chatter and prevent mechanical creep when the robot is expected to remain stationary:

$$v_t = \begin{cases} 0 & \text{if } |\bar{v}_t| < \epsilon \\ \bar{v}_t & \text{otherwise} \end{cases}, \quad \omega_t = \begin{cases} 0 & \text{if } |\bar{\omega}_t| < \epsilon \\ \bar{\omega}_t & \text{otherwise} \end{cases} \quad (10)$$

The finalized, mechanically safe velocities (v_t, ω_t) are then published to the motor drivers and cached as the previous state for the subsequent cycle.

C.3 State Estimation Comparison

Table 13: State Estimation Resource Overhead. Additional system resources consumed by the localization nodes evaluated on the NVIDIA Jetson AGX Orin 64 GB.

Odometry	CPU Usage (%)	GPU Usage (%)	Memory (MB)
ZED VIO	+19.4	+42.6	+577.3
Fast-LIVO2	+26.0	+13.3	+367.3

As discussed in Sec. 3.4 and 3.5 of the main paper, our framework requires precise and continuous odometry for two critical operations: spatial trajectory recording for failure recovery and ego-motion compensation for the reactive costmap. While the StereoLabs ZED 2i provides a built-in VIO solution, its reliance on the GPU forces it to share it with the compute-intensive VLA model, resulting in degraded inference speed. To resolve this architectural bottleneck, we strategically adopted Fast-LIVO2. By utilizing the onboard 3D LiDAR and the sensor-calibrated ZED 2i, Fast-LIVO2 fuses multimodal data (LiDAR, RGB, IMU) to achieve high precision while executing the primary odometry tasks on the CPU, as detailed in Table 13. The evaluations in Table 15 and Fig. 16 highlight the synergistic efficiency of this design. Replacing the default ZED VIO with Fast-LIVO2 protects the VLA model’s computational bandwidth, guaranteeing faster inference without sacrificing LiDAR-enhanced precision. Ultimately, as Table 14 shows, our fully integrated system (NaVILA, path deviation detection, Fast-LIVO2, and the RL policy) incurs a marginal overhead of only 0.55 s compared to standalone NaVILA. This confirms that strategically distributing odometry to the CPU and utilizing a lightweight RL policy successfully preserves the system’s overall real-time performance.

Table 14: VLA Inference Time Comparison. Evaluated over 100 runs each on the NVIDIA Jetson AGX Orin. As verified in Sec. B.4, the lightweight RL policy does not affect the VLA’s inference speed. While integrating state estimation inevitably adds overhead, Fast-LIVO2 maintains a lower latency than ZED VIO by effectively leveraging the CPU, as demonstrated in Table 2. Consequently, **Ours** (NaVILA + path deviation detection + Fast-LIVO2 + RL) requires only lightweight scalar operations from the attention heads, introducing a negligible additional overhead of ~ 0.05 s. Ultimately, with only ~ 0.55 s of total additional computation compared to standalone NaVILA, our system successfully executes primary VLA reasoning alongside trajectory recording, path deviation monitoring, and obstacle avoidance.

Configuration	Mean (s)	Median (s)	Min / Max (s)	Stdev (s)
NaVILA	3.488	3.489	3.461 / 3.509	0.008
NaVILA + RL	3.488	3.489	3.461 / 3.509	0.008
NaVILA + ZED VIO + RL	4.584	4.588	4.525 / 4.645	0.025
NaVILA + Fast-LIVO2 + RL	3.971	3.968	3.918 / 4.046	0.024
Ours	4.022	4.020	3.910 / 4.120	0.041

Table 15: State Estimation Performance Comparison. Fast-LIVO2 vs. ZED VIO Odometry evaluated over 10 indoor and 10 outdoor runs. The position and yaw errors were measured by computing the accumulated drift relative to a fixed AprilTag before and after each run. The best (lowest) error in each metric is highlighted in bold.

Environment	Method	GT Pose Error (m)		GT Yaw Error ($^{\circ}$)		Static Drift (cm)	
		Median	Mean	Median	Mean	Median	Mean
Indoor	Fast-LIVO2	0.082	0.331	0.640	4.463	0.080	0.080
	ZED VIO	0.276	0.413	1.345	5.850	0.025	0.053
Outdoor	Fast-LIVO2	0.375	0.731	6.690	13.469	0.065	0.166
	ZED VIO	0.839	4.589	8.560	22.314	0.070	0.124

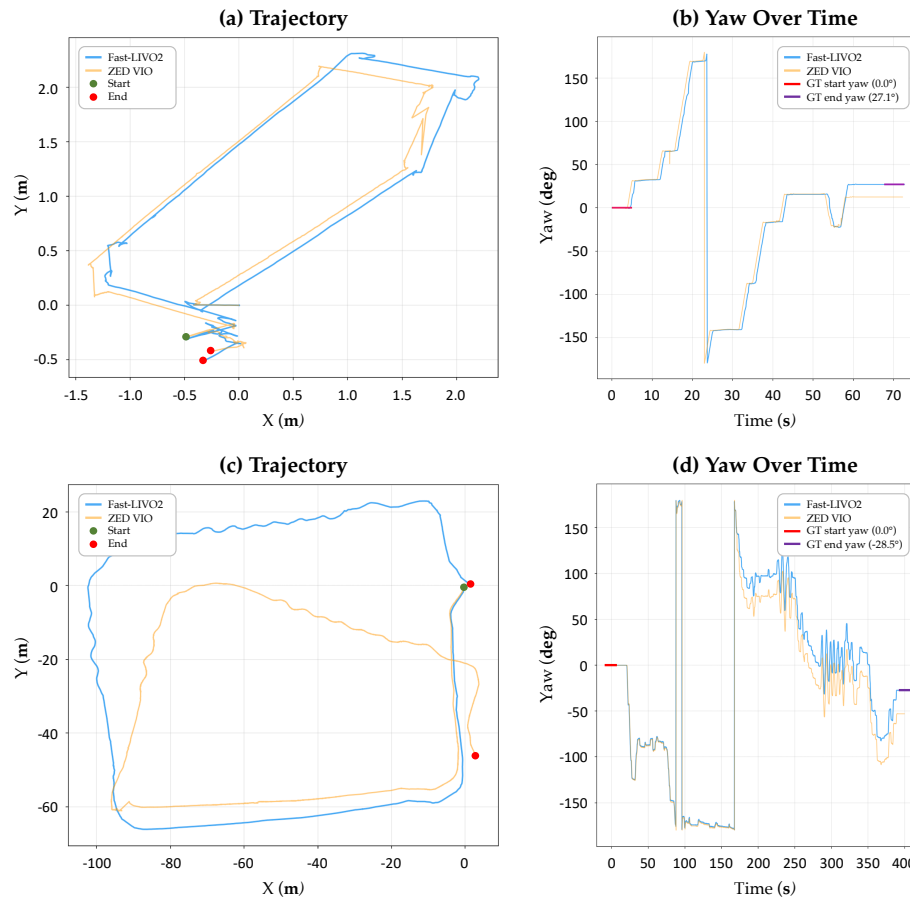


Fig. 16: Qualitative Trajectory and Yaw Estimation Analysis. Representative evaluation runs comparing **Fast-LIVO2** and **ZED VIO** in (a, b) indoor and (c, d) outdoor environments. The robot starts at the green dot and ends at the red dot. **Fast-LIVO2** successfully closes the loop in both scenarios, whereas **ZED VIO** suffers from severe scale and rotational drift, particularly in the outdoor environment.

D Real-World Deployment Results

This section presents real-world testing of our fully integrated framework, combining the VLA backbone with path deviation detection, a rollback mechanism to the saved normal trajectory, and a low-level RL avoidance policy. We highlight two key capabilities. First, Fig. 17 demonstrates robust, real-time obstacle avoidance despite naive VLA commands. Second, Fig. 18 illustrates autonomous trajectory recovery to the last normal state following a path deviation.

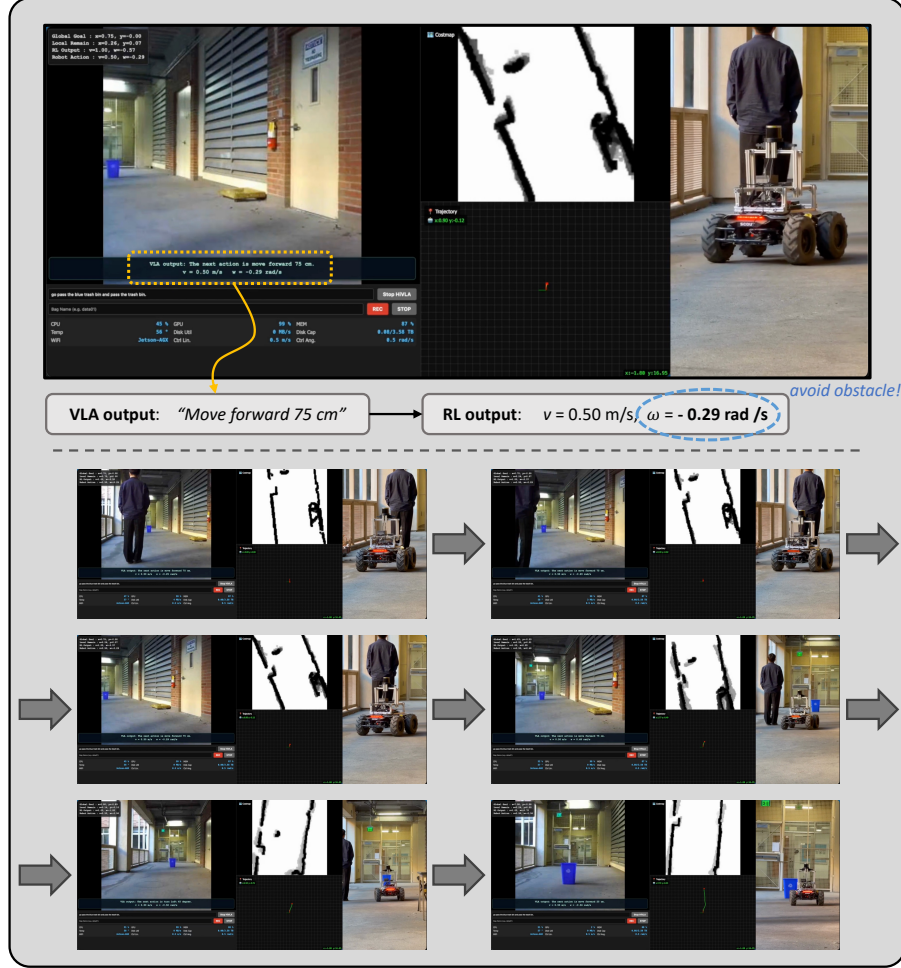


Fig. 17: Robust Obstacle Avoidance via Low-Level RL Policy. This sequence demonstrates the system’s ability to correct unsafe high-level commands in real-time. Although the VLA model naively outputs a forward movement command ("Move forward 75 cm") despite a pedestrian blocking the path, our RL policy independently detects the obstacle. It immediately overrides the naive command by executing a safe avoidance maneuver ($v=0.50 \text{ m/s}$, $\omega=0.29 \text{ rad/s}$), allowing the robot to steer around the pedestrian without collision and continue its navigation.

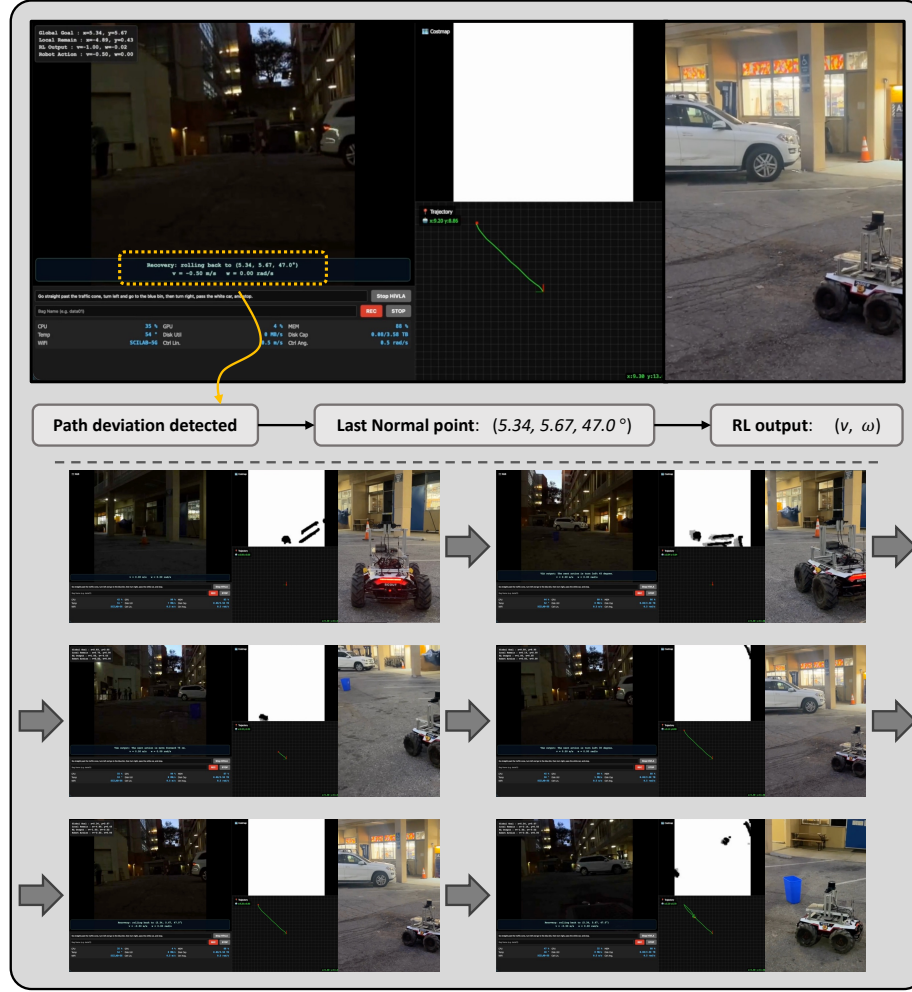


Fig.18: Path Deviation Detection and Autonomous Trajectory Recovery. This sequence demonstrates the system’s rollback mechanism. When a path deviation caused by VLA hallucination is detected during navigation, the system aborts the erroneous trajectory. It then initiates a recovery phase toward the last normal state (e.g., $x = 5.34, y = 5.67, z = 47.0^\circ$) from the real-time recorded trajectory. During this process, the robot automatically executes a direct safe recovery to that point, guided by the obstacle avoidance actions of the RL policy, ensuring the mission resumes smoothly.